Univerzita Jana Evangelisty Purkyně
v Ústí nad Labem
Přírodovědecká fakulta



Systém pro monitorování a analýzu dopravy pomocí počítačového vidění

Vypracoval: Patrik Poklop

Vedoucí práce: Ing. Roman Vaibar Ph.D., MBA

Studijní program: Aplikovaná informatika

Studijní obor: Api

Ústí nad Labem 2026

Namísto žlutých stránek vložte digitálně podepsané zadání kvalifikační práce poskytnuté vedoucím katedry.
Zadání musí zaujímat právě dvě strany.

Zadání je nutno vložit jako PDF pomocí některého nástroje, který umožňuje editaci dokumentů (se zachováním elektronického podpisu).

V Linuxe lze například použít příkaz `pdftk`.

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a použil jen pramenů, které cituji a uvádím v přiloženém seznamu literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., ve znění zákona č. 81/2005 Sb., autorský zákon, zejména se skutečností, že Univerzita Jana Evangelisty Purkyně v Ústí nad Labem má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Jana Evangelisty Purkyně v Ústí nad Labem oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladu, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Ústí nad Labem dne 8. listopadu 2026 Podpis: . . . . . .

Děkuji vedoucímu práce Ing. Romanu Vaibarovi, Ph.D., MBA
za neocenitelné rady a pomoc při tvorbě bakalářské práce.

Systém pro monitorování a analýzu dopravy pomocí počítačového vidění

Abstrakt: Tato bakalářská práce se zabývá návrhem, implementací a vyhodnocením systému pro monitorování a analýzu silniční dopravy pomocí metod počítačového vidění. Cílem práce je vytvořit funkční prototyp zařízení schopného v reálném čase analyzovat videozáznam z dopravní lokality, detekovat a klasifikovat účastníky provozu (primárně osobní automobily, autobusy a nákladní vozidla, případně chodce) a zaznamenávat jejich průjezdy definovanou oblastí zájmu (ROI). Navržený systém je realizován na platformě Raspberry Pi 5 s využitím dedikovaného AI akcelerátoru Hailo-8 a moderních detekčních modelů rodiny YOLO. Součástí práce je také softwarová aplikace pro ukládání záznamů do relační databáze a jejich interaktivní vizualizaci. Funkčnost, výpočetní výkon a přesnost systému jsou ověřeny pomocí experimentů nad reálným dopravním záznamem za zhoršených povětrnostních podmínek a porovnány s referenčním řešením na grafické kartě (GPU) i manuálním pozorováním.

Klíčová slova: počítačové vidění, monitorování dopravy, detekce objektů, Edge AI, Raspberry Pi

System for Traffic Monitoring and Analysis using Computer Vision

Abstract: This bachelor's thesis focuses on the design, implementation, and evaluation of a road traffic monitoring and analysis system using computer vision methods. The aim of the thesis is to develop a functional prototype capable of real-time video analysis of a traffic scene, including the detection and classification of traffic participants (primarily passenger cars, buses, and trucks, as well as pedestrians) and counting their movements across a defined region of interest (ROI). The proposed system is implemented on the Raspberry Pi 5 platform using the Hailo-8 AI accelerator and modern YOLO object detection models. The thesis also includes the development of a software application for relational database storage and interactive data visualization. The functionality, computational performance, and accuracy of the system are verified through experiments on real-world winter traffic footage and evaluated against both a reference GPU implementation and manual ground truth observations.

Keywords: computer vision, traffic monitoring, object detection, Edge AI, Raspberry Pi

Obsah

Seznam zkratek **13**

Úvod **17**

1. Teoretická část **19**
1.1. Sledování a počítání objektů 19
1.2. Počítačové vidění pro analýzu dopravy 25
1.3. Přehled detekčních modelů 28
1.4. Systémy pro monitorování dopravy 31
1.5. Edge AI a vestavěné systémy 34
1.6. Metodika testování a evaluace 37
1.7. Etické aspekty a GDPR 40

2. Návrh systému **43**
2.1. Požadavky na systém a volba technologií 43
2.2. Výběr hardwaru 45
2.3. Celková architektura a tok dat 45
2.4. Návrh softwarových modulů 47

3. Implementace systému **53**
3.1. Modularizace kódu, správa konfigurace a kontejnerizace 53
3.2. Příprava a kompilace detekčního modelu 54
3.3. Implementace zpracování videa a detekce 56
3.4. Komunikace, ukládání a vizualizace dat 59

4. Testování a vyhodnocení **63**
4.1. Testovací scénáře a metodické vymezení experimentu 63
4.2. Vyhodnocení přesnosti detekce a matice záměn 64
4.3. Vyhodnocení přesnosti sčítání a klasifikace 66
4.4. Vyhodnocení výkonnosti systému 68

5. Diskuze a prezentace výsledků **71**

6. Závěr **73**

A. Externí přílohy **79**

Seznam zkratek

AI	Artificial Intelligence (Umělá inteligence)
API	Application Programming Interface (Aplikační programové rozhraní)
ARM	Advanced RISC Machine (Architektura energeticky úsporných procesorů)
BCE	Binary Cross-Entropy (Binární křížová entropie)
CNN	Convolutional Neural Network (Konvoluční neuronová síť)
COCO	Common Objects in Context (Standardizovaný dataset pro detekci objektů)
CPU	Central Processing Unit (Centrální procesorová jednotka)
CSI	Critical Success Index (Index kritické úspěšnosti detekce)
DeepSORT	Deep Simple Online and Realtime Tracking (Rozšíření algoritmu SORT o hluboké učení)
DFC	Dataflow Compiler (Kompilační nástroj pro architekturu Hailo)
Edge AI	Edge Artificial Intelligence (Umělá inteligence na okraji sítě / lokální zpracování modelů na koncových zařízeních)
ER	Entity-Relationship (Entitně-relační datový model / diagram)
FN	False Negatives (Falešně negativní detekce / přehlédnuté objekty)
FP	False Positives (Falešně pozitivní detekce)
FPS	Frames Per Second (Počet zpracovaných snímků za sekundu)
GDPR	General Data Protection Regulation (Obecné nařízení o ochraně osobních údajů)
GFLOPs	Giga Floating Point Operations Per Second (Miliardy operací v plovoucí řádové čárce za sekundu)
GPU	Graphics Processing Unit (grafický procesor)
GT	Ground Truth (Referenční skutečná hodnota stanovená manuální anotací)
HAR	Hailo Archive (Meziformát archivu modelu před kvantizací)
HEF	Hailo Executable Format (Spustitelný binární formát modelu pro čipy Hailo)
HTTP	Hypertext Transfer Protocol (Hypertextový přenosový protokol)
ID	Identifier (Unikátní identifikátor sledovaného objektu)

IoT Internet of Things (Internet věcí)

IoU Intersection over Union (Metrika prostorového překryvu ohraničujících obdélníků)

IP Internet Protocol (Internetový protokol pro přenos dat po síti)

JSON JavaScript Object Notation (Textový formát pro výměnu strukturovaných dat)

mAP Mean Average Precision (Střední průměrná přesnost detekčního modelu)

MOT Multiple Object Tracking (Víceobjektové sledování)

MQTT Message Queuing Telemetry Transport (Lehký protokol pro přenos zpráv v IoT)

NMS Non-Maximum Suppression (Algoritmus pro potlačení nemaximálních překrývajících se detekcí)

NPU Neural Processing Unit (Neuronový procesor pro akceleraci umělé inteligence)

ONNX Open Neural Network Exchange (Otevřený standardizovaný formát pro výměnu neuronových sítí)

ONVIF Open Network Video Interface Forum (Globální standard pro komunikaci IP kamer)

OOP Object-Oriented Programming (Objektově orientované programování)

OS Operating System (Operační systém)

PC Personal Computer (Osobní počítač)

PCIe Peripheral Component Interconnect Express (Vysokorychlostní počítačová sběrnice)

PoE Power over Ethernet (Technologie pro napájení zařízení po datovém kabelu)

PTQ Post-Training Quantization (Post-tréninková kvantizace vah neuronové sítě)

R-CNN Region-based Convolutional Neural Network (Architektura dvoufázové detekce objektů)

RAM Random Access Memory (operační paměť s přímým přístupem)

REST Representational State Transfer (Architektonický styl pro návrh webových rozhraní)

RGB Red-Green-Blue (Aditivní barevný prostor složený z červené, zelené a modré složky)

ROI Region of Interest (Oblast zájmu / sledovaný výřez v obraze)

RTSP Real-Time Streaming Protocol (Síťový protokol pro přenos streamovaného videa)

SORT Simple Online and Realtime Tracking (Algoritmus pro online sledování v reálném čase)

SQL Structured Query Language (Strukturovaný dotazovací jazyk pro relační databáze)

SSD Single Shot MultiBox Detector (Architektura jednofázové detekce objektů)

TN True Negatives (Správně negativní případy / zamítnuté pozadí)

TOPS Tera Operations Per Second (Biliony výpočetních operací za sekundu)

TP True Positives (Správně pozitivní detekce)

UML Unified Modeling Language (Unifikovaný jazyk pro modelování softwarových systémů)

YOLO You Only Look Once (Architektura jednofázové detekce objektů)

Úvod

Rostoucí intenzita dopravy představuje jednu z hlavních výzev současných měst a aglomerací. Zvyšující se počet vozidel klade vysoké nároky nejen na propustnost samotné dopravní infrastruktury, ale také na bezpečnost silničního provozu a celkovou kvalitu života obyvatel. Efektivní řízení a plánování dopravy proto vyžaduje přesná, kontinuálně sbíraná a aktuální data. Tradiční metody získávání těchto dat, jako je například manuální sčítání dopravy, jsou však časově i personálně náročné a náchylné k chybám. Vzniká tak silná potřeba automatizace celého procesu.

V reakci na tyto potřeby a v rámci širšího rozvoje konceptu chytrých měst (z angl. *Smart Cities*) se jako vysoce efektivní řešení ukazuje využití moderních technologií. Klíčovým nástrojem se v tomto kontextu stává počítačové vidění v kombinaci s umělou inteligencí. Kamerové systémy doplněné o algoritmy strojového učení poskytují flexibilní a škálovatelné řešení pro automatizované sledování, vyhodnocování a predikci dopravních jevů. Jejich hlavní výhodou je možnost nasazení v reálném prostředí stávajících komunikací bez nutnosti rozsáhlých a nákladných stavebních zásahů do infrastruktury.

S překotným rozvojem výpočetních technologií a hardwarové miniaturizace se v této oblasti navíc stále více prosazuje přístup tzv. Edge AI (umělá inteligence na okraji sítě). Zpracování rozsáhlých obrazových dat díky tomu již nemusí probíhat ve vzdálených datacentrech, ale je realizováno přímo na okrajových zařízeních v místě sběru dat. Využití kompaktních jednodeskových počítačů doplněných o specializované hardwarové AI akcelerátory umožňuje dramatické snížení latence, zvýšení ochrany citlivých osobních údajů a minimalizaci závislosti na stabilním internetovém připojení či cloudových službách. V dopravních aplikacích, kde je vyžadováno spolehlivé zpracování v reálném čase, je tento přístup zásadní.

Cílem této bakalářské práce je navrhnout, implementovat a otestovat plně funkční systém pro automatické sčítání a analýzu dopravního provozu, postavený na principech Edge AI. Práce se zaměřuje na detekci, sledování a klasifikaci různých účastníků provozu – primárně osobních automobilů, autobusů a nákladních vozidel, případně chodců a cyklistů v definované lokalitě s využitím moderních detekčních modelů pro víceobjektové sledování. Důraz je kladen na praktickou realizaci celého řešení na dostupném okrajovém zařízení a na následné vyhodnocení přesnosti detekce i sčítání z reálných videozáznamů dopravy.

Text práce je strukturován do šesti hlavních kapitol. První, teoretická část poskytuje ucelený přehled v oblastech počítačového vidění, metod detekce a sledování objektů a existujících systémů pro monitorování dopravy. Následně se věnuje specifikům Edge AI a etickým aspektům sběru dat. Druhá kapitola přibližuje celkový návrh architektury systému, včetně zdůvodnění výběru

hardwarových komponent. Třetí kapitola detailně popisuje samotnou implementaci řešení od přípravy vývojového prostředí až po ukládání a vizualizaci nasbíraných dat. Čtvrtá a pátá kapitola jsou dedikovány testování systému v reálných scénářích, vyhodnocení přesnosti modelu a diskuzi nad dosaženými výsledky. Závěrečná kapitola pak shrnuje zjištěné poznatky a nastiňuje možnosti budoucího rozvoje vytvořené aplikace.

1. Teoretická část

Tato kapitola shrnuje teoretická východiska a klíčové metody potřebné pro návrh a realizaci automatizovaného systému dopravního monitoringu. V úvodních sekcích jsou rozebrány principy víceobjektového sledování (MOT), stavové predikce pomocí Kalmanova filtru a prostorové metody sčítání průjezdů využívající oblasti zájmu (ROI). Navazující část se věnuje předzpracování obrazových dat a evoluci hlubokých konvolučních neuronových sítí (CNN) pro jednofázovou detekci objektů v reálném čase, přičemž hlavní důraz je kladen na architekturu rodiny YOLO a porovnání jejích velikostních variant. Závěr kapitoly zasazuje tyto algoritmy do kontextu moderních inteligentních dopravních systémů, vysvětluje principy výpočtů na okraji sítě (Edge AI) s využitím hardwarových akcelerátorů (NPU), definuje exaktní matematické metriky pro evaluaci přesnosti a vymezuje legislativní požadavky ochrany soukromí dle nařízení GDPR.

1.1. Sledování a počítání objektů

Sledování objektů v obrazových sekvencích a videu představuje klíčovou nadstavbu nad samotnou detekcí objektů. Zatímco detekce umožňuje lokalizovat a klasifikovat objekty v jednotlivých izolovaných snímcích, sledování zajišťuje jejich konzistentní identifikaci napříč časem. Tato vlastnost je vysoce heterogenní a zásadní zejména v dopravních aplikacích, kde je pro komplexní analýzu nutné sledovat plynulý pohyb vozidel, určovat jejich trajektorie, rychlost nebo měřit přesný počet průjezdů daným úsekem komunikace [1, 2].

Víceobjektové sledování (MOT)

Víceobjektové sledování (MOT, z angl. *Multiple Object Tracking*) se zabývá současným sledováním většího počtu dynamických objektů ve scéně. Hlavním cílem je přiřazení unikátních a časově konzistentních identifikátorů (Unique ID) každému objektu při jeho prvním výskytu ve scéně. Tento identifikátor je následně udržován po celou dobu výskytu entity ve sledované sekvenci, což umožňuje spojování detekcí mezi jednotlivými snímky a rekonstrukci trajektorie objektu v čase. V dopravních monitorovacích systémech nachází MOT uplatnění zejména při analýze dopravních toků, odhadu hustoty provozu, detekci anomálií nebo sledování pohybu vozidel a chodců na komplexních křižovatkách [2].

Tento proces je ve většině moderních architektur realizován pomocí přístupu „detekce před sledováním“ (z angl. *Tracking-by-Detection*). Systém nejprve v každém snímku identifikuje objekty

pomocí detekčního konvolučního modelu a následně provádí asociaci těchto detekcí mezi po sobě jdoucími snímky na základě kombinace prostorových, časových a vzhledových charakteristik. Správná správa těchto identifikátorů je kritická pro následné navazující úlohy, jako je statistické počítání objektů nebo pokročilá analýza jejich chování [1].



Obrázek 1.1.: Ukázka víceobjektového sledování vozidel s přiřazenými časově konzistentními identifikátory (vlastní zpracování, akcelerátor Hailo-8)

Jak je znázorněno na obrázku 1.1, implementovaný systém umožňuje stabilní sledování většího množství objektů v reálném čase, přičemž každému vozidlu zůstává zachováno jeho unikátní číslo i při pohybu napříč snímanou scénou.

Algoritmy pro sledování objektů

Mezi nejpoužívanější a nejefektivnější přístupy pro sledování objektů v reálném čase patří algoritmy SORT a DeepSORT. Tyto metody jsou oblíbené zejména pro svou nízkou výpočetní náročnost, vysokou propustnost a snadnou integraci s moderními detekčními modely, jako je rodina modelů YOLO [3].

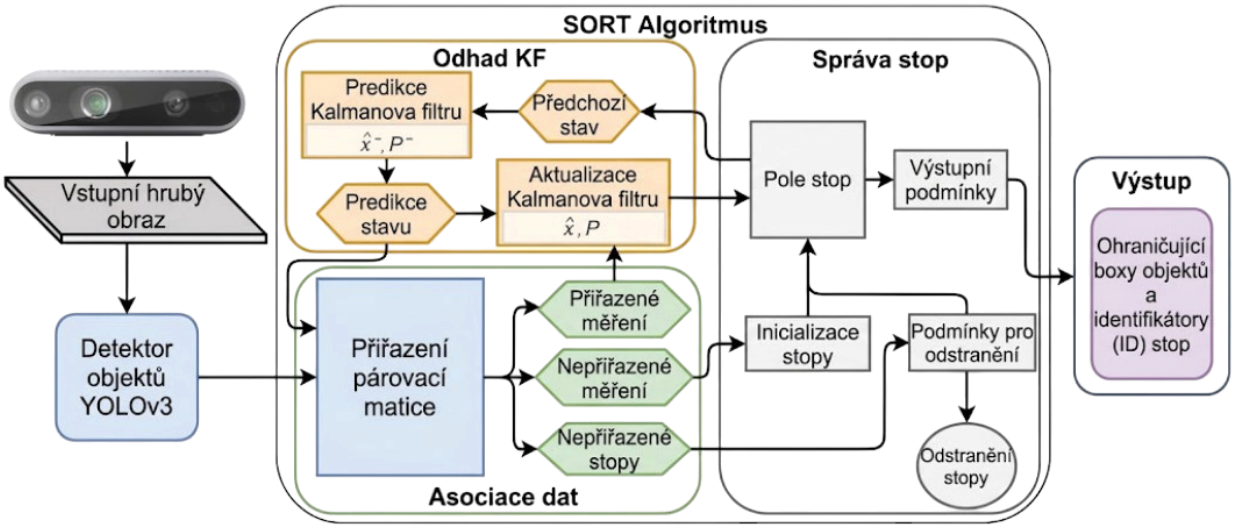
SORT

Algoritmus SORT (z angl. *Simple Online and Realtime Tracking*) je postaven na kombinaci dvou klíčových komponent: Kalmanova filtru[1] a Maďarského algoritmu pro datovou asociaci. Kalmanův filtr slouží k predikci budoucí polohy objektu v následujícím snímku na základě lineárního

[1]Kalmanův filtr je rekurzivní matematický algoritmus, který ze série nepřesných a zašuměných měření (v tomto případě historických detekcí z kamery) odhaduje aktuální a predikuje budoucí stav dynamického systému (například polohu, směr a rychlost jedoucího vozidla).

pohybového modelu. Stav každého sledovaného objektu je definován vektorově pomocí souřadnic středu ohraničujícího obdélníku, jeho plochy (měřítka), poměru stran a jejich příslušných rychlostí změn v čase.

Jakmile jsou v novém snímku získány čerstvé detekce, nastupuje Maďarský algoritmus, který řeší optimalizační úlohu přiřazení predikovaných poloh k reálným novým detekcím. Jako nákladová funkce (metrika vzdálenosti) se zde využívá překryv ohraničujících obdélníků vyjádřený pomocí metriky (IoU, z angl. *Intersection over Union*). Díky této matematické přímočarosti dosahuje SORT extrémní rychlosti zpracování, avšak jeho zásadní nevýhodou je nestabilita při dlouhodobějších překryvech (okluzích) objektů, kdy absence vizuální informace často vede ke ztrátě stopy a fatálnímu selhání přiřazení ID [4].



Obrázek 1.2.: Celkový přehled a princip fungování sledovacího algoritmu SORT (vlastní překlad, adaptováno z [5, s. 5]).

Na obrázku 1.2 je detailně znázorněna celá pracovní smyčka (z angl. *pipeline*) algoritmu SORT [4], která se nezávisle opakuje s každým novým obrazovým snímkem. Podle blokového schématu lze celý proces logicky rozdělit do tří na sebe navazujících fází:

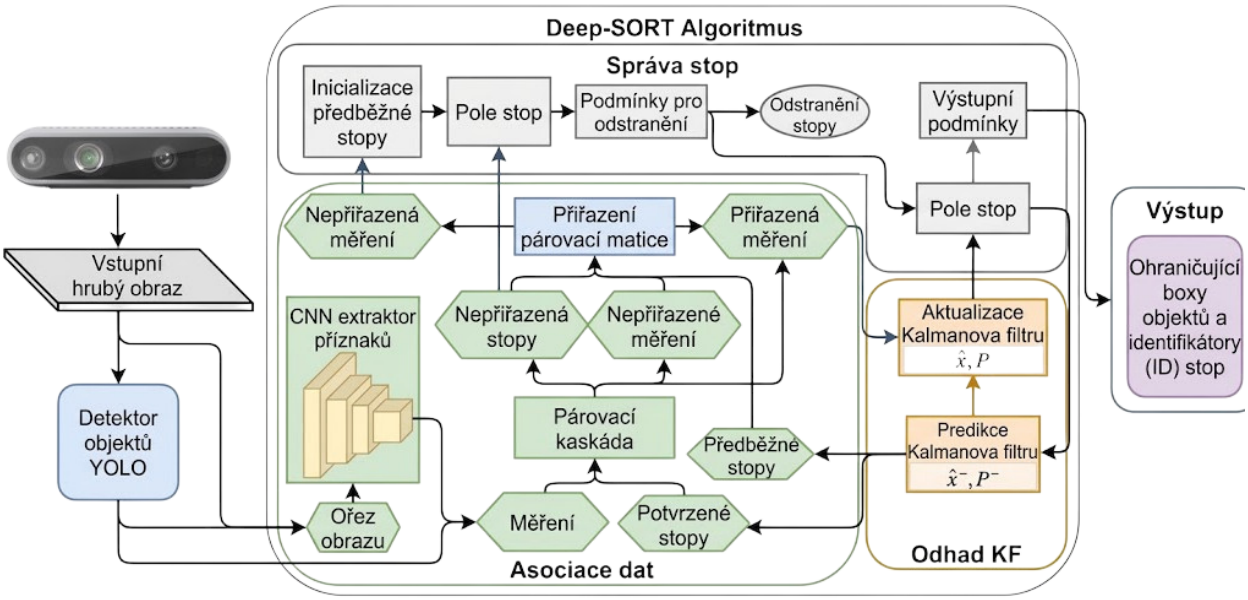
1. **Odhad KF:** Tato část schématu představuje paměť systému. Na základě předchozího stavu (*Předchozí stav*) algoritmus provede matematickou predikci (*Predikce Kalmanova filtru*), čímž získá odhadovanou polohu sledovaných objektů v aktuálním čase (*Predikce stavu*).
2. **Asociace dat:** Do tohoto bloku vstupují jak odhadnuté polohy z Kalmanova filtru, tak zcela nové detekce z modelu YOLO. Úkolem párovací matice (*Přiřazení párovací matice*) je optimálně spojit tyto dva vstupy pomocí metriky IoU. Výstupem jsou tři možné stavy: úspěšně spárovaná měření (*Přiřazené měření*), nové detekce bez historie (*Nepřiřazené měření*) a existující stopy, ke kterým se nenašla nová detekce (*Nepřiřazené stopy*).
3. **Správa stop:** V závěrečné fázi systém reaguje na výsledky předchozí asociace. Spárovaná měření aktualizují Kalmanův filtr a udržují se v poli aktivních stop (*Pole stop*). Z nespárovaných měření, což typicky představuje nově příchozí vozidla, se inicializují stopy nové

(*Inicializace stopy*). Nespárované stopy, tedy vozidla, která pravděpodobně opustila scénu, jsou po splnění stanovených podmínek ze systému odstraněny (*Odstranění stopy*). Validní stopy jsou následně propuštěny na finální výstup (*Výstup*).

DeepSORT

Algoritmus DeepSORT (z angl. *Deep Simple Online and Realtime Tracking*) vznikl jako přímá reakce na slabá místa původního algoritmu SORT v komplexních a hustých scénách. Rozšiřuje čistě geometrické a prostorové přiřazování o integraci hlubokých konvolučních neuronových sítí, které jsou vytrénovány pro extrakci vizuálních vzhledových příznaků (z angl. *deep appearance descriptors*). Pro každý detekovaný objekt je vygenerován charakteristický vektor (embedding), který reprezentuje jeho vizuální identitu.

Při samotné asociaci dat pak DeepSORT kombinuje dvě metriky: Mahalanobisovu vzdálenost, která reflektuje pohybovou konzistenci na základě Kalmanova filtru, a kosinovou vzdálenost, jež vyhodnocuje shodu vizuálních příznaků s historií daného objektu. Propojení těchto informací probíhá pomocí kaskádového párování (*matching cascade*). Tento kombinovaný přístup dramaticky zvyšuje robustnost sledování v hustém dopravním provozu, kde dochází k častému vzájemnému zakrývání vozidel, a prokazatelně minimalizuje množství chyb typu záměny identity (*ID switches*) [6].



Obrázek 1.3.: Blokové schéma architektury sledovacího algoritmu DeepSORT a jeho rozhodovacích kroků (vlastní překlad, adaptováno z [5, s. 7])

Na obrázku 1.3 je detailně znázorněna architektura a datový tok algoritmu Deep-SORT. Podle blokového schématu lze celý proces rozdělit do následujících klíčových fází, které se vykonávají s každým novým snímkem:

1. **Extrakce příznaků a predikce polohy:** Vstupní obraz (*Vstupní hrubý obraz*) je nejprve zpracován detektorem objektů, ze kterého jsou získány obrazové výřezy jednotlivých vozidel

(*Ořez obrazu*). Tyto výřezy následně zpracovává konvoluční neuronová síť (*CNN extraktor příznaků*), jež extrahuje unikátní vizuální příznaky. Souběžně s tím modul odhadu (*Odhad KF*) provádí matematickou predikci budoucí polohy pomocí Kalmanova filtru (*Predikce Kalmanova filtru*).

2. **Asociace dat:** Tento centrální modul kombinuje nová měření obohacená o vizuální příznaky (*Měření*) a potvrzené stopy z minulosti (*Potvrzené stopy*). Klíčovým inovačním prvkem oproti původnímu algoritmu SORT je kaskádové párování (*Párovací kaskáda*), které prioritně využívá právě vizuální podobnost. Objekty a měření, které kaskáda nedokáže spojit (*Nepřiřazené stopy* a *Nepřiřazená měření*), jsou zachyceny a předány do klasické párovací matice (*Přiřazení párovací matice*), jež funguje jako záloha využívající standardní prostorový překryv.
3. **Správa stop a aktualizace:** Výstupy z asociace dat se dělí do tří směrů. Úspěšně spárovaná měření (*Přiřazená měření*) jsou využita k okamžité aktualizaci Kalmanova filtru (*Aktualizace Kalmanova filtru*). Zcela nová detekovaná vozidla, která nemají historii (*Nepřiřazená měření*), zakládají v systému nové, zatím nepotvrzené stopy (*Inicializace předběžné stopy*). Správa stop následně kontroluje celé pole aktivních záznamů (*Pole stop*). Vozidla, jež dlouhodobě chybí a splní podmínky pro smazání (*Podmínky pro odstranění*), jsou ze systému odstraněna (*Odstranění stopy*). Stopy splňující výstupní kritéria (*Výstupní podmínky*) jsou předány na finální výstup v podobě souřadnic a unikátních identifikátorů (*Výstup*).

ByteTrack

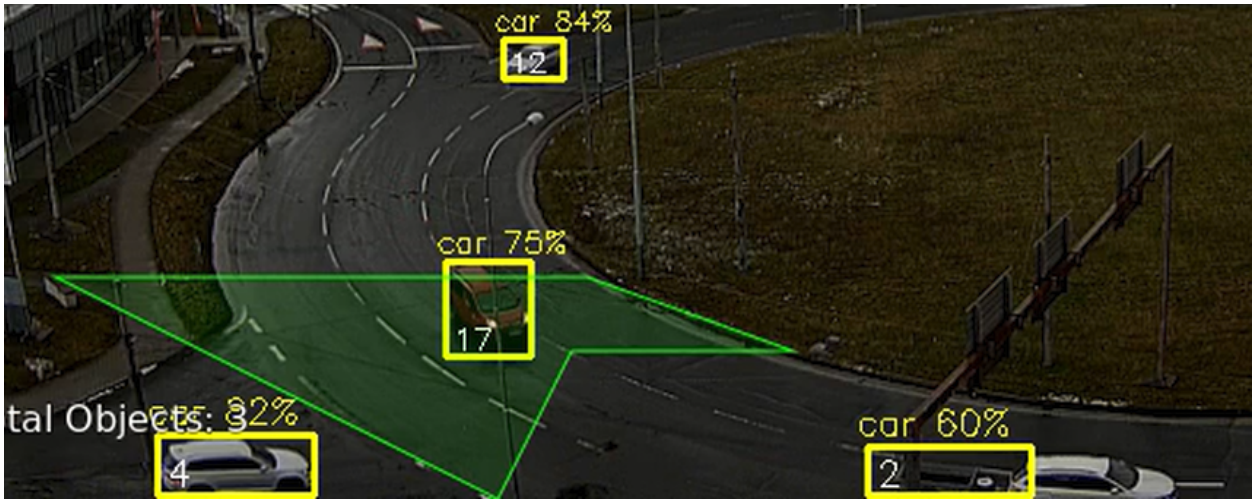
Algoritmus ByteTrack [7] představuje moderní rozšíření algoritmu SORT [4], které řeší problém ztráty sledované stopy při částečném zákrytu vozidla, aniž by vyžadovalo výpočetně náročnou extrakci vzhledových příznaků pomocí konvoluční sítě jako DeepSORT [6]. Zatímco tradiční přístupy zahazují detekční rámečky, jejichž míra spolehlivosti (*confidence*) klesne pod stanovený práh, ByteTrack zavádí dvoufázové párování. V první fázi spáruje existující stopy s vysoce spolehlivými detekcemi nad prahem a ve druhé fázi asociuje zbylé stopy i se slabšími detekcemi (které mají nižší spolehlivost například kvůli dočasnému zakrytí jiným vozidlem či hustým sněžením). Díky absenci pomocné neuronové sítě dosahuje ByteTrack vysoké snímkové propustnosti (FPS) v reálném čase [7].

Metody sčítání objektů

Samotné automatické sčítání objektů v dopravním proudu bezprostředně navazuje na výsledky získané z MOT. Namísto prosté kumulace detekcí v jednotlivých samostatných snímcích, která by vedla k masivnímu nadhodnocení výsledků (například z důvodu vícenásobného započítání téhož stojícího či pomalu jedoucího vozidla), se využívá analýzy časoprostorových trajektorií jednoznačně identifikovaných objektů [2].

V praxi se pro automatizované sčítání a kategorizaci využívají dva primární koncepční přístupy:

- **Virtuální čára (z angl. *Virtual Line*):** Jedná se o nejrozšířenější a výpočetně nejméně náročnou metodu pro sčítání vozidel v konkrétních jízdních pruzích či směrech. V obrazovém prostoru je definována úsečka fungující jako detekční brána. Systém v každém kroku kontroluje, zda vektor trajektorie konkrétního objektu (typicky reprezentovaný středovým nebo spodním bodem jeho ohraničujícího obdélníku) protnul tuto linii. V okamžiku detekce průsečíku je inkrementováno počítadlo pro daný směr jízdy. Tento přístup je vysoce efektivní pro stanovení okamžité intenzity dopravy na liniových úsecích komunikací [2].
- **Oblast zájmu (ROI, z angl. *Region of Interest*):** Alternativní přístup spočívá ve vymezení uzavřené polygonální nebo obdélníkové zóny přímo v digitálním obraze. Sledovaný objekt je zaregistrován a statisticky započítán buď v momentě vstupu do této definované oblasti, nebo při jejím opuštění. Tento princip se využívá především v situacích, kde je nutné sledovat celkovou obsazenost určitého vyhrazeného prostoru (např. plochy křižovatek, odbočovací pruhy nebo parkoviště) či analyzovat průměrnou dobu zdržení vozidel v daném segmentu [1, 2].



Obrázek 1.4.: Definice analytické oblasti zájmu (ROI) pomocí polygonu v praktické implementaci sledovacího systému (vlastní zpracování, akcelerátor Hailo-8).

Jak ukazuje obrázek 1.4, zatímco na rovných úsecích komunikací často postačuje k detekci průjezdu jednoduchá virtuální čára, složitější dopravní uzly, jakými jsou kruhové objezdy, vyžadují robustnější přístup. V rámci předložené implementace je proto oblast zájmu (ROI) definována jako nepravidelný n-úhelník (polygon). Tento geometrický útvar je v systému konstruován pomocí přesné sady 2D souřadnic (vrcholů) definovaných v obrazovém prostoru předzpracovaného snímku.

Proces samotného sčítání a vyhodnocování trajektorií je následně založen na průběžném výpočtu prostorového průniku (z angl. *spatial intersection*). Systém v každém snímku kontinuálně extrahuje polohu sledovaných objektů, jež jsou typicky reprezentovány středovým bodem (centroidem) jejich ohraničujícího obdélníku. Jakmile vyhodnocovací algoritmus detekuje, že se tento referenční bod vozidla nachází přímo uvnitř definované polygonové zóny (což matematicky odpovídá úloze

bodu v polygonu, tzv. *Point-in-Polygon*), je vozidlo okamžitě započítáno do celkové statistiky dopravy. Tento moment je na obrázku názorně zachycen u objektu s identifikátorem 17. Tento plošný přístup výrazně eliminuje chyby spojené s nepřesnostmi detekce na hranách a umožňuje spolehlivé sčítání i při zakřivených a překrývajících se trajektoriích.

1.2. Počítačové vidění pro analýzu dopravy

Počítačové vidění (z angl. *Computer Vision*) představuje interdisciplinární oblast informatiky, která se zabývá automatickou analýzou statických snímků a videozáznamů. Jeho hlavním cílem je extrakce relevantních a sémantických informací z vizuálních vstupů. V oblasti moderních dopravních systémů je počítačové vidění využíváno zejména v aplikacích dopravního dohledu, proaktivního řízení provozu a při automatické analýze dopravní propustnosti. Kamerové systémy umožňují kontinuální sledování dopravních toků, detekci anomálních situací nebo identifikaci konkrétních vozidel a jejich registračních značek [1, 2].

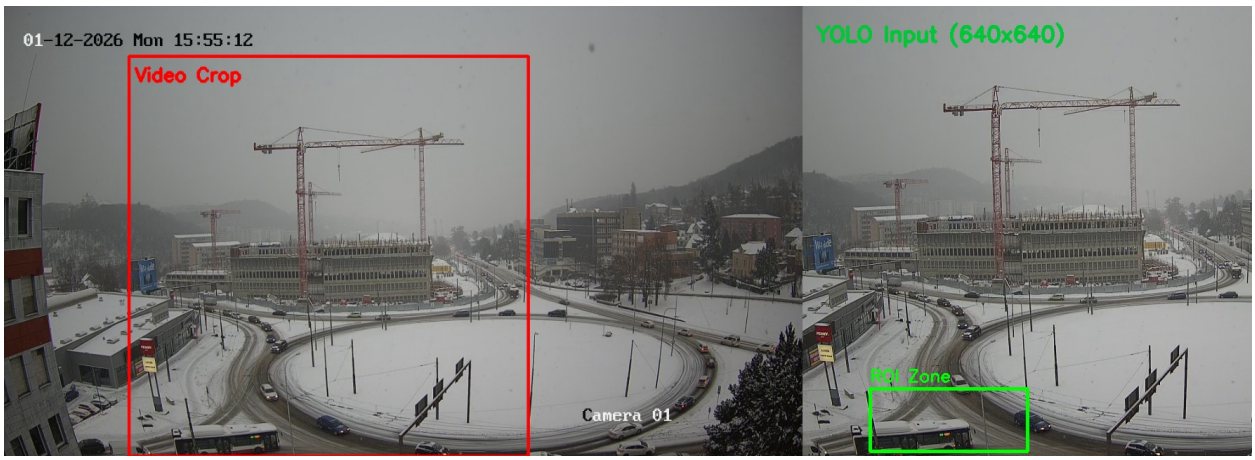
Předzpracování obrazu

Předzpracování obrazu představuje nezbytný přípravný krok před samotnou aplikací detekčních modelů. Cílem je zlepšení kvality vstupních dat, potlačení digitálního šumu a optimalizace celkové výpočetní zátěže.

V dopravních aplikacích je surový videozáznam z kamery často výpočetně příliš objemný a obsahuje velké množství irelevantních informací (např. oblohu, okolní budovy, chodníky). Prvním logickým krokem je proto definice oblasti zájmu (ROI). Softwarové oříznutí nebo zavedení masky zaručí, že systém analyzuje pouze samotnou vozovku. To nejen radikálně snižuje objem zpracovávaných dat, ale také eliminuje falešné detekce, jako jsou například zaparkovaná vozidla mimo zorné pole křižovatky.

Následně musí být upravený obraz transformován do formátu, který striktně vyžaduje architektura neuronové sítě. Jedná se primárně o **škálování (z angl. *resizing*)** na fixní rozlišení (typicky 640×640). Aby při změně velikosti nedošlo k deformaci poměru stran, což by zhoršilo schopnost sítě rozeznat tvar a proporce vozidla, využívá se běžně technika zvaná *letterboxing*. Ta zmenší obraz proporcionálně a chybějící prostor (okraje) doplní neutrální barvou.

Poslední rutinní fází je **normalizace barev**. Hodnoty jednotlivých pixelů, které standardně nabývají celočíselných hodnot 0 až 255, jsou převedeny na desetinná čísla v intervalu 0,0 až 1,0. Tento krok stabilizuje a výrazně urychluje matematické výpočty (násobení matic) uvnitř umělé neuronové sítě [1]. I přes tuto důkladnou datovou optimalizaci však analýza obrazu zůstává nesmírně výpočetně náročná, což představuje hlavní výzvu při nasazení na okrajových zařízeních.



Obrázek 1.5.: Příklad dvoufázového vymezení oblasti zájmu, hardwarový ořez vlevo a softwarová analytická zóna vpravo (vlastní zpracování).

Jak ukazuje obrázek 1.5, předzpracování obrazového toku hraje klíčovou roli v optimalizaci celého systému. Namísto výpočetně náročnějších metod (jako je doplňování černých pruhů u *letterboxingu*) je v navrženém systému využit přímý prostorový ořez obrazu (z angl. *crop*). Levá část obrázku znázorňuje zdrojový snímek, který byl v rámci datovodu nejprve naškálován na rozlišení 1280×720 pixelů. Červený obdélník v něm přesně vymezuje hranice hardwarového ořezu ten je definován počátečními souřadnicemi $x = 200, y = 80$ s pevnou šířkou a výškou 640 pixelů.

Pravá část následně představuje fyzicky vyříznutý čtverec, jenž svými rozměry (640×640) nativně odpovídá požadavkům vstupní vrstvy modelu YOLO. Je nutné zdůraznit, že tento hardwarový ořez slouží výhradně k obrazové přípravě dat před vstupem do modelu. Samotná analytická oblast zájmu (ROI), která určuje, kde reálně dochází k detekci a sčítání vozidel, je v systému definována až následně. Na pravé části obrázku je tato aplikační logika vizualizována zeleným obdélníkem. Jeho poloha je v aplikačním kódu dynamicky vypočítávána pomocí relativních souřadnic vůči velikosti předzpracovaného výstupu (v tomto konkrétním případě $x = 96, y = 544, w = 224, h = 90$). Oddělením těchto dvou fází je do neuronové sítě propuštěn čistší datový tok, přičemž analytika si zachovává plnou flexibilitu.

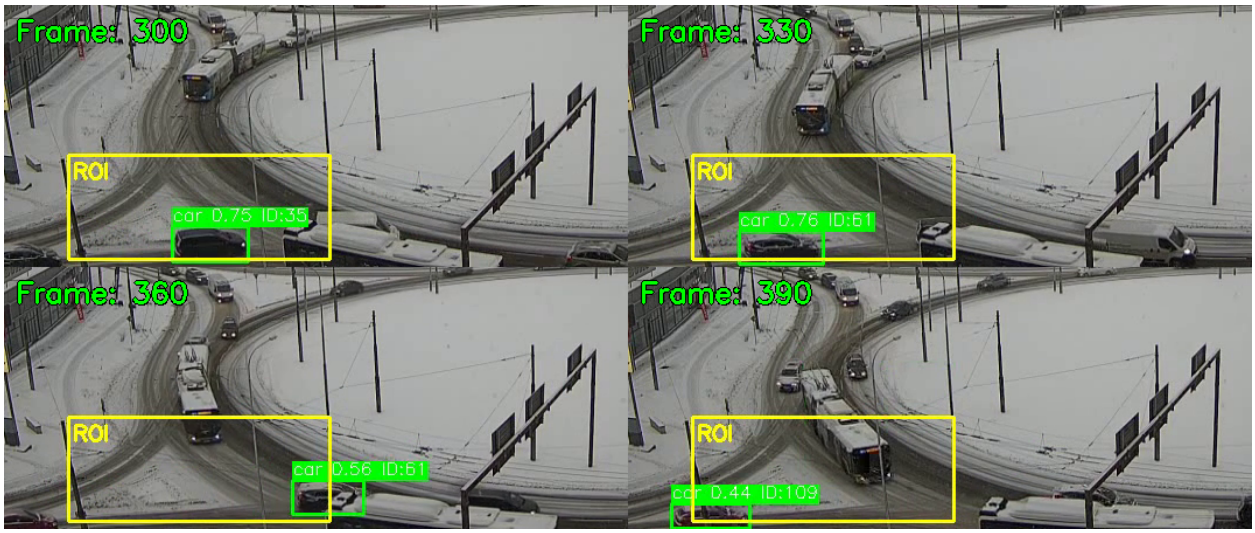
Základní úlohy počítačového vidění

V rámci analýzy dopravy a návrhu inteligentní dopravní infrastruktury se počítačové vidění opírá o tři základní, na sebe úzce navazující úlohy:

- **Detekce objektů:** Zaměřuje se na samotnou lokalizaci objektů v obraze. Výstupem této úlohy jsou obvykle ohraničující obdélníky (z angl. *bounding boxes*), které vymezují prostorovou polohu nalezených entit. V dopravních aplikacích se jedná především o detekci vozidel, chodců nebo cyklistů.
- **Klasifikace objektů:** Představuje úlohu, při které je již detekovanému objektu přiřazena předem definovaná třída na základě jeho vizuálních charakteristik. V praxi se využívá

například pro rozlišení, zda je detekované vozidlo osobní automobil, nákladní vůz nebo autobus [1].

- **Sledování objektů:** Umožňuje analyzovat pohyb detekovaných a klasifikovaných objektů napříč po sobě jdoucími snímky video sekvence. Slouží k odhadu trajektorií, predikci směru a výpočtu rychlosti pohybu [2].



Obrázek 1.6.: Časová sekvence čtyř snímků demonstrující kontinuální sledování objektů a vyhodnocování jejich polohy vůči definované zóně (ROI). Systém úspěšně udržuje ID u projíždějícího automobilu a dynamicky registruje nová vozidla (vlastní zpracování, inference na GPU).

Jak je názorně ukázáno na obrázku 1.6, propojení úlohy detekce a sledování umožňuje systému přidělit jednotlivým vozidlům unikátní identifikátory (ID) a kontinuálně je sledovat v čase. Obrázek zachycuje sekvenci čtyř po sobě jdoucích snímků s pevným rozestupem 30 rámečků (snímky 300, 330, 360 a 390). Vzhledem k tomu, že zdrojové video je zpracováváno frekvencí 15 snímků za sekundu (FPS), představuje časový krok mezi každým zobrazeným dílčím obrázkem přesně 2 sekundy reálného času (celá mřížka tak mapuje celkem 6 sekund dopravního provozu). Pro každé detekované vozidlo systém neustále vypočítává středový bod (centroid) jeho ohraničujícího obdélníku. Zásadní analytickou podmínkou je, že vozidlo je do statistik průjezdů aktivně zahrnuto pouze tehdy, nachází-li se tento vypočítaný střed fyzicky uvnitř definované oblasti zájmu (žlutý obdélník ROI).

Plynulý průjezd a spolehlivost sledování jsou prokazatelně demonstrovány na osobním automobilu (označeném jako *car ID:61*). Toto vozidlo je zachyceno ve sledované zóně ROI ve snímku 330, přičemž o dvě sekundy později (snímek 360) si systém navzdory jeho pohybu napříč křižovatkou úspěšně udržuje jeho původní ID. Z mřížky je rovněž jasně patrné, jak algoritmus dynamicky reaguje na celkovou obměnu vozidel ve scéně. Zatímco na prvním snímku (300) zónou projíždí předchozí vozidlo s ID 35, na posledním snímku (390) do ROI zleva vstupuje další automobil, kterému systém okamžitě a správně přiděluje nové, dosud nepoužité ID (109). Tento holistický

přístup, kombinující striktní prostorové omezení analytickou zónou a konzistentní sledování identity napříč snímky, je v dopravních aplikacích naprosto klíčový pro zabránění falešnému vícenásobnému započítání téhož objektu.

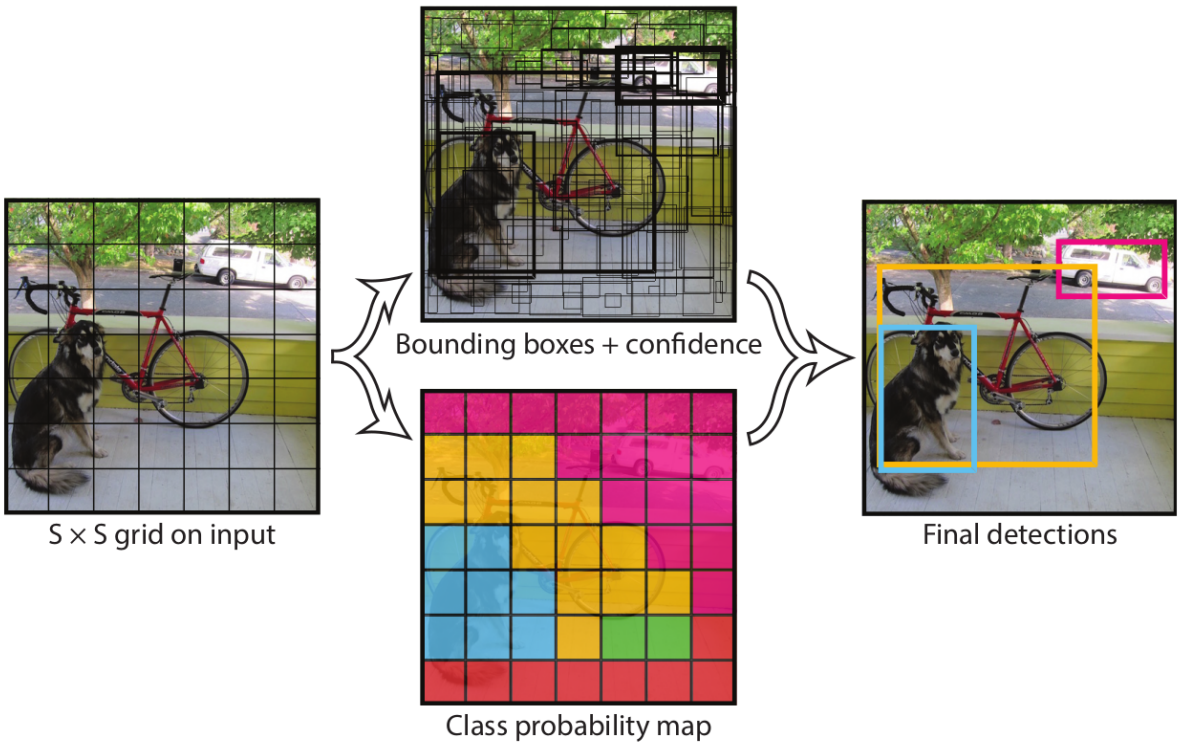
1.3. Přehled detekčních modelů

V posledních letech dominují oblasti detekce objektů architektury hlubokých konvolučních neuronových sítí (CNN). Pro nasazení na výpočetně omezených okrajových zařízeních se historicky osvědčily zejména modely z rodiny SSD a YOLO, které patří do kategorie tzv. jednofázových detektorů (z angl. *one-stage detectors*). Tyto modely dokáží obraz zpracovat během jediného průchodu sítí, čímž splňují výše zmíněné požadavky na nízkou latenci.

Architektura SSD (z angl. *Single Shot MultiBox Detector*) detekuje objekty tak, že využívá hustou sadu předem definovaných kotevních obdélníků (z angl. *anchor boxes*) o různých poměrech stran a velikostech. Predikce probíhá na několika mapách příznaků s různým rozlišením současně, což modelu teoreticky umožňuje lépe zachytit objekty nejrůznějších velikostí [1].

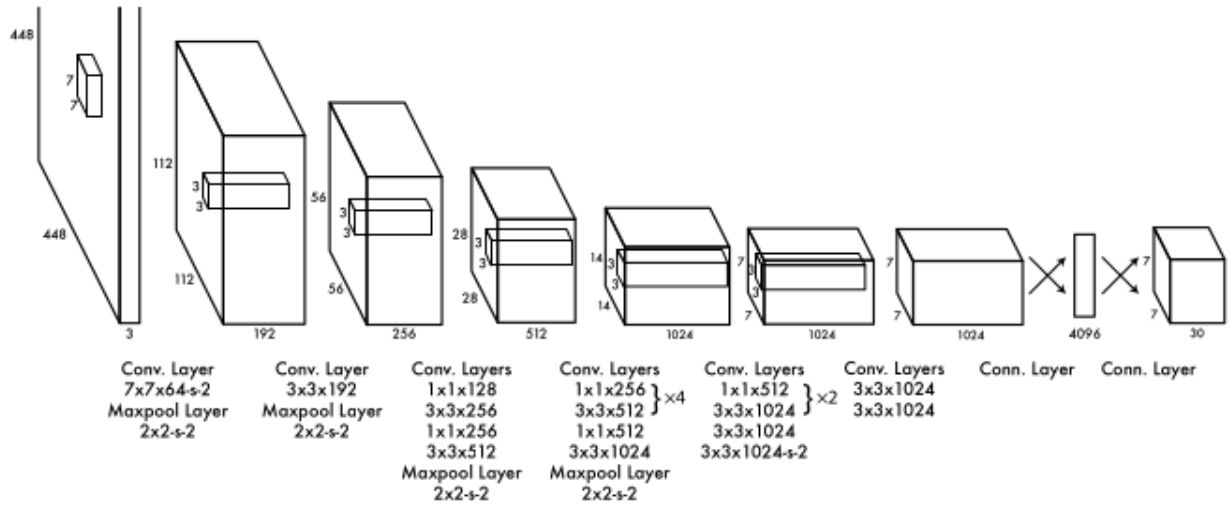
Model YOLO

Ačkoliv SSD dosahuje stabilní přesnosti, naprostou revoluci v efektivitě přinesla architektura YOLO (z angl. *You Only Look Once*), která od základů transformovala přístup k počítačovému vidění. Namísto dvoufázového generování návrhů oblastí (jako u starších sítí R-CNN, z angl. *Region-based Convolutional Neural Network*) formuluje detekci jako jedinou, *end-to-end* regresní úlohu [3].



Obrázek 1.7.: Základní princip detekce modelu YOLO využívající rozdělení obrazu do mřížky (převzato z [3, s. 2])

Jak ukazuje obrázek 1.7, vstupní obraz je nejprve virtuálně rozdělen do mřížky $S \times S$. Pokud střed hledaného objektu spadne geometricky do konkrétní buňky této mřížky, stává se tato konkrétní buňka plně zodpovědnou za jeho detekci. Každá buňka následně predikuje fixní počet B ohraničujících obdélníků, míru spolehlivosti (z angl. *confidence score*) pro tyto obdélníky a pravděpodobnosti zařazení do C různých tříd. Celkový výstup sítě je tak elegantně zakódován do jediného tenzoru o rozměrech $S \times S \times (B \cdot 5 + C)$.



Obrázek 1.8.: Vnitřní architektura modelu YOLO znázorňující sekvenci konvolučních a plně propojených vrstev (převzato z [3, s. 3])

Na obrázku 1.8 je detailně znázorněna topologie vrstev původního modelu YOLO, jejíž architektura byla přímo inspirována modelem GoogLeNet[2]. Systém na svém vstupu přijímá obraz o přesném rozlišení 448×448 pixelů. Samotná síť se skládá z 24 konvolučních vrstev, které slouží k postupné a hluboké extrakci vizuálních příznaků z obrazu (od jednoduchých hran až po komplexní tvary vozidel). Za nimi následují 2 plně propojené vrstvy (z angl. *fully connected layers*), jež získané příznaky agregují a generují finální predikční tenzor o rozměrech $7 \times 7 \times 30$.

Klíčovým inovačním prvkem této architektury je zařazení střídavých redukčních konvolučních vrstev s filtrem 1×1. Tyto vrstvy fungují jako informační úzká hrdla (z angl. *bottlenecks*) – efektivně zmenšují prostor příznaků z předchozích vrstev, čímž drasticky redukují množství potřebných matematických operací bez zásadní ztráty kontextu. Moderní varianty modelů z rodiny YOLO jsou pro účely přesné objektové detekce standardně trénovány a vyhodnocovány na rozsáhlém datasetu COCO[3] (z angl. *Common Objects in Context*) [8]. Tento dataset obsahuje stovky tisíc detailně anotovaných snímků reálných scén a plně definuje 80 základních tříd objektů, což detekčnímu modelu poskytuje vysoce robustní výchozí základ pro spolehlivé rozpoznávání různorodých účastníků dopravního provozu (osobní automobily, autobusy, nákladní vozy či chodce).

Architektura a škálování variant modelu YOLOv8

Významný architektonický posun přinesla generace modelů **YOLOv8**, vyvinutá společností Ultralytics [9, 10]. Oproti starším generacím opouští YOLOv8 koncept předdefinovaných kotevních obdélníků a zavádí plně bezkotevní (*anchor-free*) detekci, která přímo predikuje střed objektu a jeho vzdálenosti k hranám ohraničujícího rámečku. Detekční hlava je navíc rozdělena (*decoupled head*) na dvě nezávislé větve – jedna řeší výhradně klasifikaci tříd pomocí binární křížové entropie (BCE) a druhá prostorovou regresi souřadnic pomocí ztrátových funkcí CIoU a DFL (*Distribution Focal Loss*). V páteřní síti (*backbone*) a krku (*neck*) byl starší konvoluční blok C3 nahrazen novým stavebním modulem **C2f** (*CSP Bottleneck with 2 convolutions*), který propojuje příznaky z více mezivrstev a zajišťuje bohatší tok gradientů při zachování nízké výpočetní náročnosti [9].

Aby bylo možné tutéž architekturu nasadit jak na extrémně úsporných mikropočítačích, tak na výkonných cloudových serverech, je rodina YOLOv8 rozdělena do pěti velikostních variant: **Nano (`n`)**, **Small (`s`)**, **Medium (`m`)**, **Large (`l`)** a **Extra Large (`x`)** [10]. Všechny tyto varianty sdílejí totožnou makro-architekturu (stejné pořadí konvolučních vrstev, bloků C2f a vrstvy SPPF), ale liší se hodnotou tří škálovacích koeficientů (*compound scaling constants*) [9, 10]:

- **Hloubkový multiplikátor (d, `depth_multiple`):** Určuje, kolikrát se uvnitř každého modulu C2f opakují vnitřní reziduální bloky (*Bottleneck*). Tím přímo řídí celkový počet vrstev neuronové sítě – menší modely `YOLOv8n` a `YOLOv8s` ($d = 0{,}33$) čítají v neslitém výpočetním grafu **225 vrstev** (168 vrstev po fúzi konvoluce a dávkové normalizace při inferenci), střední

[2] GoogLeNet (známý také pod označením Inception v1) je průlomová architektura hluboké konvoluční neuronové sítě od společnosti Google, která v roce 2014 zvítězila v prestižní soutěži počítačového vidění ImageNet (ILSVRC). Vyznačuje se velmi vysokou výpočetní efektivitou.

[3] COCO dataset

model `YOLOv8m` ($d = 0{,}67$) obsahuje **295 vrstev** (218 po fúzi) a největší modely `YOLOv8l` a `YOLOv8x` ($d = 1{,}00$) dosahují hloubky **365 vrstev** (268 po fúzi).

- **Šířkový multiplikátor (w, `width_multiple`):** Škáluje počet konvolučních filtrů (výstupních kanálů) v každé vrstvě sítě v rozsahu od $w = 0{,}25$ (varianta `n`) až po $w = 1{,}25$ (varianta `x`). Růst šířky sítě kvadraticky zvyšuje počet trénovatelných parametrů i nároky na výpočetní výkon (GFLOPs).
- **Maximální počet kanálů ($c_{\max}$, `max_channels`):** Omezuje horní hranici počtu kanálů v nejhlubších vrstvách sítě (1 024 kanálů pro `n`/`s`, 768 pro `m` a 512 pro `l`/`x`), aby u hlubokých modelů nedocházelo k neúměrnému nárůstu paměťové stopy.

Srovnání všech pěti variant při vstupním rozlišení 640×640 pixelů na referenčním datasetu COCO val2017 shrnuje tabulka 1.1.

Varianta	Hloubka (d)	Šířka (w)	Počet vrstev (fúze)	Parametry	Výpočty (FLOPs)	mAP$_{50-95}$
YOLOv8n	0,33	0,25	225 (168)	3,2 M	8,7 GFLOPs	37,3 %
YOLOv8s	0,33	0,50	225 (168)	11,2 M	28,6 GFLOPs	44,9 %
YOLOv8m	0,67	0,75	295 (218)	25,9 M	78,9 GFLOPs	50,2 %
YOLOv8l	1,00	1,00	365 (268)	43,7 M	165,2 GFLOPs	52,9 %
YOLOv8x	1,00	1,25	365 (268)	68,2 M	257,8 GFLOPs	53,9 %

Tabulka 1.1.: Teoretické porovnání architektury a parametrů variant modelu YOLOv8 při vstupním rozlišení 640×640 px na datasetu COCO val2017 [9, 10].

Jak vyplývá z tabulky 1.1, přechod od nejmenší varianty `YOLOv8n` ke střední variantě **`YOLOv8m`** (zvolené pro implementaci na akcelerátoru Hailo-8 v této práci) přináší skokové zvýšení detekční přesnosti mAP$_{50-95}$ z 37,3 % na 50,2 % díky hlubší extrakci příznaků v 295 vrstvách. Naopak další zvětšování modelu na varianty `l` a `x` již přináší pouze mírný nárůst přesnosti (o 2,7 až 3,7 procentního bodu), avšak za cenu dvojnásobného až trojnásobného nárůstu výpočetní složitosti (GFLOPs), což je pro okrajová zařízení s omezenou pamětí neefektivní.

1.4. Systémy pro monitorování dopravy

Monitorování dopravy představuje klíčovou součást moderních inteligentních dopravních systémů. Jeho primárním cílem je kontinuální získávání informací o dopravním provozu, jako je intenzita dopravy, průměrná rychlost vozidel, detekce kongescí nebo výskyt nestandardních dopravních událostí. Tyto agregované informace jsou následně využívány pro aktivní řízení dopravy, strategické plánování městské infrastruktury a zvyšování celkové bezpečnosti silničního provozu [2].

Tradiční metody dopravního monitoringu

Tradiční metody monitorování dopravy jsou historicky založeny převážně na fyzických kontaktních senzorech nebo manuálním sběru dat. Mezi jejich hlavní výhody patří relativně přímočará implementace a dlouhodobé ověření v praxi, zásadní nevýhodou je však jejich omezená flexibilita, neschopnost poskytovat hlubší kontext a vysoké náklady na instalaci i následnou údržbu.

Jednou z nejstarších metod je manuální sčítání dopravy, při kterém jsou dopravní prostředky zaznamenávány lidským pozorovatelem. Tento přístup je extrémně časově i personálně náročný, náchylný k lidským chybám způsobeným únavou a je naprosto nevhodný pro kontinuální dlouhodobý monitoring dopravního provozu [2].

Další široce rozšířenou technologií jsou indukční smyčky zabudované přímo do povrchu vozovky, které detekují přítomnost kovového podvozku vozidla na základě změny elektromagnetického pole. Ačkoliv poskytují relativně spolehlivá data o prostých průjezdech vozidel, jejich instalace vyžaduje destruktivní zásah do vozovky, což omezuje dopravu. Samotné smyčky navíc podléhají rychlému opotřebení vlivem zátěže a povětrnostních podmínek [2].

Pro monitorování rychlosti a vzdálenosti jsou rovněž hojně využívány radarové a lidarové senzory. Tyto technologie dosahují vysoké přesnosti i za zhoršené viditelnosti, avšak jejich nevýhodou jsou vysoké pořizovací náklady a omezené možnosti vizuální klasifikace jednotlivých typů objektů (např. obtížné rozlišení mezi malým motocyklem a chodcem) [2].

Moderní kamerové systémy

S raketovým rozvojem počítačového vidění a dostupností vysokého výpočetního výkonu se v dopravní infrastruktuře stále častěji prosazují kamerové systémy založené na analýze obrazových dat. Na rozdíl od jednorozměrných dat z indukčních smyček umožňují kamerové systémy získávat z jednoho zařízení bohaté, vícerozměrné informace o celé dopravní scéně, včetně přesného typu vozidel, jejich trajektorií nebo anomálního chování účastníků provozu [1].

Mezi hlavní výhody těchto systémů patří vysoká flexibilita (změna sledované oblasti často vyžaduje pouze softwarovou úpravu virtuálních čar, nikoliv překopání vozovky) a možnost využití jedné kamery pro paralelní běh více analytických úloh. Nevýhodou kamerových systémů jsou naopak jejich limity spojené s kvalitou vstupních obrazových dat. Výkon detekčních a sledovacích algoritmů může být negativně ovlivněn prudkými změnami světelných podmínek, nepříznivým počasím (mlha, hustý déšť) nebo vzájemným vizuálním překrýváním objektů ve scéně. Tyto faktory představují výzvu zejména v hustém městském provozu [2].

Vstupní zařízení kamerových systémů

Pro zajištění kvalitního obrazového vstupu, který je pro algoritmy počítačového vidění kritický, se v moderních systémech Edge AI využívají primárně dva typy snímacích zařízení, jež přímo reflektují potřeby vývoje i reálného nasazení v dopravní infrastruktuře:

- **IP kamery:** Představují současný standard v budování chytrých měst. Tato zařízení nejenže snímají obraz, ale rovnou ho digitálně kódují a odesílají prostřednictvím počítačové sítě (nejčastěji pomocí protokolů RTSP nebo ONVIF). Jejich obrovskou výhodou je snadná integrace do existující síťové infrastruktury a možnost napájení přes jeden datový kabel (technologie PoE). Pro účely detekce na okrajových zařízeních poskytují IP kamery ideální a stabilní zdroj datového toku, ze kterého systém v reálném čase extrahuje jednotlivé snímky pro inferenci.
- **Kamerové moduly s přímým rozhraním (např. senzory Sony IMX):** Jedná se o kompaktní kamerové moduly navržené pro přímé hardwarové připojení k jednodeskovým počítačům (jako je platforma Raspberry Pi) pomocí vysokorychlostní sběrnice MIPI CSI. Zásadní výhodou tohoto řešení je minimální latence, jelikož surová obrazová data putují ze snímače rovnou do operační paměti výpočetního zařízení bez nutnosti jakéhokoliv síťového kódování a dekódování. Tyto senzory představují vysoce efektivní nástroj pro prototypování a autonomní nasazení analytických uzlů bez nutnosti budování složité lokální sítě.



Obrázek 1.9.: Typické umístění dohledových IP kamer na sloupu infrastruktury. Toto neinvazivní řešení poskytuje stabilní obrazový tok pro následnou detekci a inferenci na okrajových zařízeních (převzato z [11]).

Existující řešení a projekty

V současnosti existuje na trhu řada komerčních i výzkumných řešení zaměřených na monitorování dopravy. Většinu těchto systémů lze z architektonického hlediska rozdělit na dvě hlavní větve: centralizovaná cloudová řešení a distribuovaná řešení na okrajích sítě (Edge AI).

Komerční platformy často spoléhají na to, že kamerové systémy na křižovatkách slouží pouze jako pasivní snímače, které odesílají nepřetržitý videostream do vzdálených cloudových datových center. Zde se za pomoci masivního výpočetního výkonu provádí přesná analytika a trasování. Ačkoliv je tento přístup vysoce přesný, naráží na limity v podobě vysokých nároků na propustnost síťového připojení, latenci a v neposlední řadě představuje bezpečnostní riziko z hlediska ochrany osobních údajů (GDPR), jelikož citlivé videozáznamy plynule opouštějí lokální infrastrukturu.

Moderním a stále preferovanějším trendem jsou proto systémy implementované přímo na okrajových zařízeních (z angl. *edge devices*). Zde je kladen důraz na efektivní využití úsporných výpočetních prostředků a AI akcelerátorů přímo v místě sběru dat [12].

1.5. Edge AI a vestavěné systémy

Koncept tzv. Edge AI označuje moderní architektonický přístup, při kterém jsou výpočetně náročné úlohy umělé inteligence realizovány přímo na okrajových zařízeních v těsné blízkosti zdroje dat, nikoliv v centrálním cloudovém prostředí. Tento přesun logiky umožňuje dramatické snížení latence, omezení objemu přenášených dat a mnohem efektivnější využití dostupných síťových zdrojů [12].

Požadavky na zpracování s nízkou latencí

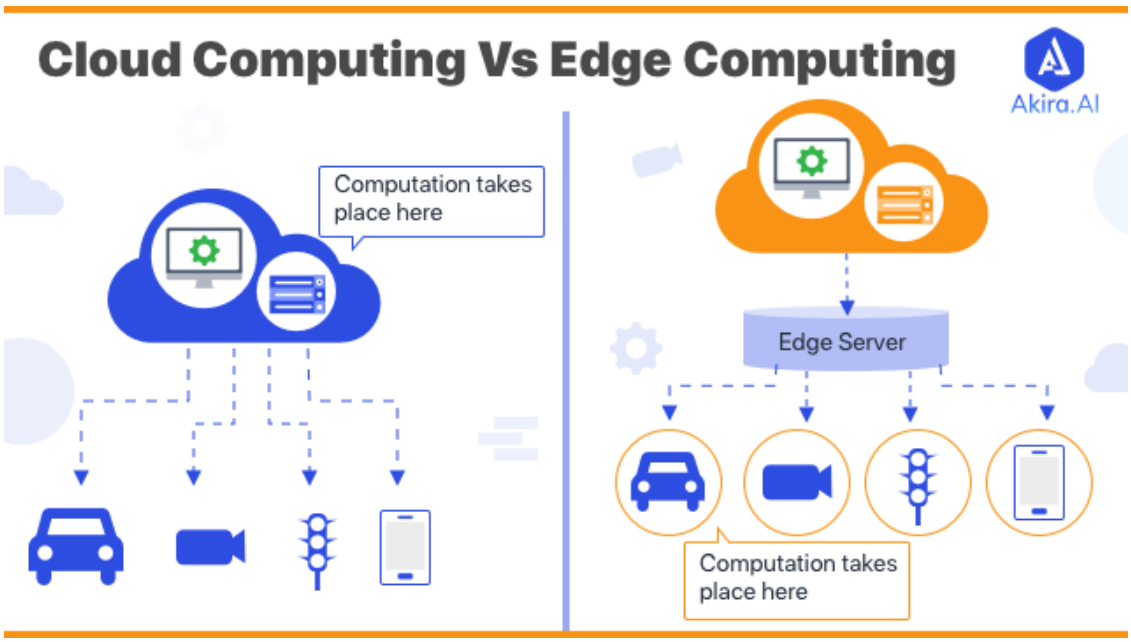
Při nasazení v reálné dopravní infrastruktuře je kladen extrémní důraz na rychlost vyhodnocování a efektivní využití dostupných hardwarových prostředků. Namísto obecného a často zavádějícího pojmu „reálný čas“ je v kontextu této práce definován přísný výkonnostní požadavek – propustnost alespoň 20 snímků za sekundu (FPS), což odpovídá průměrné době zpracování jednoho snímku kratší než 50 ms.

Tyto časové limity jsou kritické zejména u vestavěných systémů a okrajových zařízení (z angl. *edge devices*), jež nedisponují masivní výpočetní silou velkých datacenter. Splnění těchto limitů zaručuje, že nedojde ke ztrátě informací o rychle jedoucích vozidlech ani k nežádoucímu hromadění zpožděných dat ve vyrovnávacích pamětech [13].

Zpracování obrazu na okraji sítě

Zpracování obrazu na okraji sítě (z angl. *Edge Computing*) představuje přímou alternativu ke klasickému centralizovanému zpracování. V případě tradičního cloudového řešení musí být surová obrazová data (často tvořící masivní datový tok) neustále odesílána do vzdáleného datového centra. Naproti tomu edge přístup analyzuje video přímo na zařízení, ke kterému je kamera připojena.

Hlavním rozdílem a zároveň největším benefitem je eliminace zpoždění sítě (latence). Lokální inference hlubokých neuronových sítí přímo na okrajovém zařízení nejenže zajišťuje okamžitou odezvu, ale zároveň razantně zvyšuje celkovou robustnost systému. V případě dočasného výpadku internetového připojení může systém nadále plnohodnotně fungovat, lokálně ukládat agregovaná textová data o dopravě a odeslat je až po obnovení spojení [12].



Obrázek 1.10.: Porovnání architektury a datových toků u edge a cloudového zpracování (převzato z [14])

Na obrázku 1.10 je schematicky znázorněn rozdíl mezi oběma přístupy. Cloud computing se spoléhá na centrální servery, což vytváří úzké hrdlo na síťové vrstvě. Edge computing naopak přesouvá výpočetní logiku (inferenci) až na samotný okraj sítě k senzorům, což garantuje vysokou propustnost a okamžité vyhodnocování.

Hardwarové AI akcelerátory

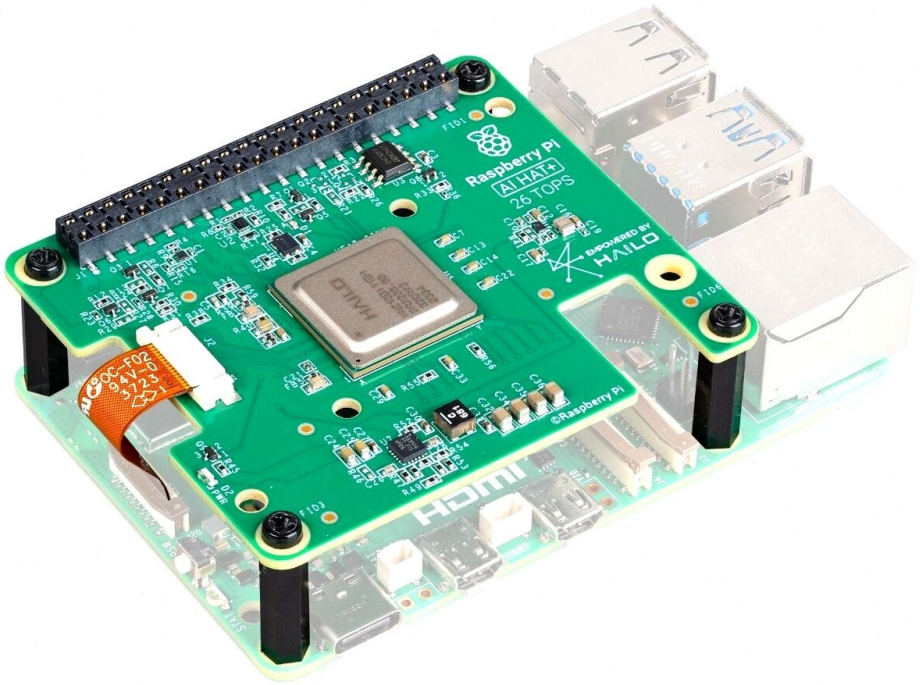
AI akcelerátory, často označované jako NPU (z angl. *Neural Processing Unit*), jsou specializované integrované obvody navržené primárně pro efektivní provádění tenzorových výpočtů. Na rozdíl od univerzálních centrálních procesorů (CPU) jsou tyto čipy na hardwarové úrovni optimalizovány pro masivní paralelní zpracování maticových operací, jež tvoří matematický základ hlubokých konvolučních sítí.

V kontextu Edge AI jsou tyto akcelerátory naprosto nezbytné, protože překonávají tzv. von Neumannovo úzké hrdlo. Běžné procesory ztrácejí obrovské množství času a energie neustálým přesouváním mezivýsledků mezi výpočetními jednotkami a hlavní pamětí. Akcelerátory tento problém eliminují tím, že udržují data co nejblíže samotným výpočetním jádrům. Umožňují tak nasazení složitých detekčních a sledovacích modelů na energeticky úsporných a prostorově omezených vestavěných zařízeních, a to při zachování propustnosti nutné pro plynulou analýzu dopravy bez závislosti na výkonných serverových farmách [13, 15].

Platforma Raspberry Pi a akcelerace pomocí Hailo-8

Platforma Raspberry Pi [4] představuje rodinu kompaktních jednodeskových počítačů (z angl. *Single-Board Computers*), které se staly standardem pro prototypování a vývoj vestavěných systémů. Její hlavní vlastností a výhodou je nízká pořizovací cena, obrovská komunitní podpora, stabilní operační systém (založený na linuxovém jádře) a především široká škála hardwarových rozhraní pro připojení periferních senzorů a modulů [16].

Zásadním omezením samotné architektury Raspberry Pi je však její hrubý výpočetní výkon. Integrovaný procesor postavený na architektuře ARM není schopen v reálném čase zpracovávat náročné úlohy počítačového vidění, a už vůbec ne zajišťovat běh hlubokých neuronových sítí. Z tohoto důvodu je pro nasazení v systémech monitorování dopravy nezbytné doplnit základní desku o externí výpočetní modul [17].



Obrázek 1.11.: Rozšiřující modul Raspberry Pi AI HAT+ s integrovaným procesorem Hailo-8 (převzato z [18])

Řešením tohoto výpočetního omezení je využití specializovaných rozšiřujících desek, jako je modul Raspberry Pi AI HAT+ (znázorněný na obrázku 1.11). Srdcem tohoto modulu, připojeného přes rozhraní PCIe, je pokročilý AI procesor Hailo-8 od stejnojmenné izraelské společnosti, jež se specializuje na vývoj výkonných neurálních akcelerátorů.

[4] Rpishop

Klíčovým parametrem tohoto akcelerátoru je deklarovaný výpočetní výkon 26 TOPS (z angl. *Tera Operations Per Second*). Předpona „Tera“ v matematickém vyjádření představuje hodnotu 10^{12}. Tento čip je tedy fyzicky schopen odbavit masivních 26×10^{12} (26 bilionů) operací za jedinou sekundu při inferenci modelu. To, jakým způsobem tohoto výkonu dosahuje, je však stejně důležité jako samotná číselná hodnota. Čipy Hailo využívají inovativní architekturu řízenou datovým tokem (z angl. *Structure-Defined Dataflow Architecture*). Zatímco tradiční GPU zpracovávají vrstvy neuronové sítě sekvenčně a neustále zapisují data do externí paměti, Hailo-8 mapuje celou topologii neuronové sítě přímo na své hardwarové zdroje. Obrazová data tak čipem protékají kontinuálně od jedné vrstvy ke druhé bez nutnosti přístupu k externí RAM.

Tato obrovská výpočetní propustnost spojená s extrémní energetickou účinností neposkytuje pouze dostatek výkonu pro plynulý běh pokročilých modelů rodiny YOLO. Systému dodává takovou kapacitní rezervu, že dokáže inferenci vysoce paralelizovat. V praxi to znamená, že systém může přijímat a vyhodnocovat obrazová data z několika nezávislých video streamů zcela naráz (například synchronní analýza RTSP toků ze dvou různých IP kamer na křižovatce), aniž by došlo k degradaci počtu zpracovaných snímků za sekundu FPS [13, 15]. Tím se toto kompaktní hardwarové řešení stává vysoce škálovatelným nástrojem pro robustní a přesné sledování dopravy.

1.6. Metodika testování a evaluace

Vyhodnocení kvality systémů pro detekci, sledování a počítání objektů představuje nedílnou součást návrhu a implementace řešení založených na počítačovém vidění. Cílem evaluace je objektivně posoudit spolehlivost navrženého systému a identifikovat jeho limity. V oblasti dopravního monitorování je však důležité odlišit hodnocení samotného matematického modelu (zda správně detekuje obdélníky v izolovaném snímku) od hodnocení celého systému jako funkčního celku (zda spolehlivě počítá vozidla v čase) [1, 2].

Metodika vyhodnocení detekce v reálném prostředí

Pro vyhodnocení přesnosti rozpoznávání objektů nasazeným modelem YOLO v konkrétním kamerovém streamu se využívá metoda manuální obrazové analýzy vybraných referenčních snímků. Vzhledem k absenci předem připraveného a anotovaného datasetu pro danou lokaci jsou z videozáznamu extrahovány reprezentativní statické snímky (např. v různých světelných podmínkách či při různé hustotě provozu).

V těchto izolovaných snímcích je následně provedeno pečlivé manuální sečtení a označení všech reálně přítomných vozidel lidským pozorovatelem, čímž vzniká lokální referenční vzorek (*ground truth*). Výstup detekčního modelu na stejném snímku je s tímto vzorkem vizuálně porovnán, přičemž jsou pro každou třídu identifikovány a zaznamenány následující hodnoty:

- **True Positives (TP):** Správně detekovaná vozidla. Model označil ohraničujícím boxem vozidlo tam, kde se skutečně fyzicky nachází.

- **False Positives (FP):** Falešné detekce. Model označil za vozidlo něco, co ním není (např. stín, značku, sloup), případně vytvořil duplicitní překrývající se detekci pro jedno a to samé auto.
- **False Negatives (FN):** Nerozpoznaná vozidla. Vozidlo na snímku prokazatelně je, ale model jej vůbec nedetekoval.
- **True Negatives (TN):** Správně zamítnuté pozadí. Tento stav označuje prázdný prostor, který model správně ignoroval. V kontextu detekce objektů ohraničujícími boxy se však hodnota TN v praxi nepočítá.

Vztah mezi predikcí modelu a skutečností se ve strojovém učení standardně vizualizuje pomocí tzv. matice záměn (z angl. *Confusion Matrix*), která je pro přehlednost znázorněna v tabulce 1.2.

Predikce/Skutečnost	**Vozidlo je přítomno**	**Vozidlo není přítomno**
Model hlásí detekci	TP	FP
Model nehlásí detekci	FN	TN

Tabulka 1.2.: Základní matice záměn (Confusion Matrix).

Na základě těchto hodnot jsou následně vypočítány reálné metriky pro danou scénu. Tato analýza poskytuje kritický vhled do toho, do jaké míry případné chyby ve finálním sčítání pocházejí z nepřesné primární detekce a do jaké míry jsou způsobeny až selháním sledovacího algoritmu.

Metriky hodnocení detekčních modelů

Při vývoji a vyhodnocování detekčních modelů i navazujících sčítacích systémů se k objektivnímu měření výkonu vůči referenčním datům (*ground truth*) využívá standardizovaná sada matematických metrik [1]:

Precision (Přesnost): Vyjadřuje podíl skutečně správných detekcí vůči všem detekcím, které systém vyprodukoval. Indikuje, do jaké míry lze modelu věřit, když ohlásí nález vozidla:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Úplnost): Udává schopnost systému nalézt všechny relevantní objekty přítomné ve scéně. Definuje se jako podíl správně detekovaných objektů vůči reálnému počtu existujících objektů:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-skóre: Představuje harmonický průměr přesnosti (Precision) a úplnosti (Recall), který posky-

tuje vyváženou souhrnnou metriku i při nerovnoměrném zastoupení chyb typu FP a FN:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

IoU (Intersection over Union): Vyjadřuje exaktní míru geometrického překryvu mezi predikovaným ohraničujícím obdélníkem (B_{pred}) a referenčním obdélníkem (B_{gt}). Čím více se hodnota blíží k jedné, tím je lokalizace přesnější:

$$\text{IoU} = \frac{\text{Plocha průniku } (B_{\text{pred}} \cap B_{\text{gt}})}{\text{Plocha sjednocení } (B_{\text{pred}} \cup B_{\text{gt}})}$$

mAP (Mean Average Precision): Představuje komplexní ukazatel výkonu modelu napříč všemi detekovanými třídami (N) na trénovacích datasetech (např. COCO), který zohledňuje přesnost, úplnost i kvalitu lokalizace zjištěnou pomocí plochy pod křivkou *Average Precision* (AP_i):

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} AP_i$$

Accuracy (Přesnost klasifikace): Metrika vyjadřující podíl správných rozhodnutí vůči všem případům. Při vyhodnocování detekce dopravních objektů (kde se neurčuje počet TN prázdného pozadí, tedy $TN = 0$) přechází vztah do podoby tzv. *Critical Success Index*:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

FPS (Frames Per Second) a doba zpracování snímku: Pro vyhodnocení výpočetní výkonnosti a schopnosti provozu v reálném čase se u detekčních a sledovacích systémů standardně měří snímková propustnost FPS, tedy počet snímků zpracovaných za jednu sekundu [2, 3]. Určuje se jako podíl celkového počtu zpracovaných snímků (N_{frames}) a celkové doby běhu výpočetního řetězce (T_{total}). Převrácená hodnota propustnosti ($\bar{t}_{\text{frame}}$) pak udává průměrný časový krok potřebný ke zpracování jednoho snímku v milisekundách:

$$\text{FPS} = \frac{N_{\text{frames}}}{T_{\text{total}}} \quad [\text{s}^{-1}], \qquad \bar{t}_{\text{frame}} = \frac{1000}{\text{FPS}} \quad [\text{ms}]$$

V zřetězených (*pipelined*) architekturách je nutné tento průměrný krok zpracování snímku ($\bar{t}_{\text{frame}}$) odlišovat od celkové koncové (*end-to-end*) latence živého přenosu, která navíc zahrnuje prodlevu síťové vyrovnávací paměti přijímače (RTSP bufferu) a dekódování videa [19].

Vzhledem k tomu, že v rámci této bakalářské práce je implementován a nasazován již hotový předtrénovaný model z rodiny YOLO, není primárním cílem optimalizace jeho interních vah pomocí trénovacích metrik (jako je mAP) na rozsáhlých testovacích datasetech. Cílem je naopak ověření jeho reálné snímkové propustnosti a praktické přesnosti jakožto sčítacího softwarového uzlu v konkrétním dopravním prostředí.

Metodika vyhodnocení počítání v reálném prostředí

Vyhodnocení správnosti sčítání objektů je v dopravních aplikacích klíčové. Na rozdíl od detekce izolovaných snímků se zde hodnotí konzistence sledovacího algoritmu (např. zda udrží ID vozidla) a správná prostorová identifikace průjezdů vyhrazenými počítacími liniemi.

Hlavními zdroji chyb v reálném prostředí jsou časté vizuální překryvy objektů (okluze), ztráta identifikátoru způsobující opakované započítání téhož vozidla, nebo dynamické změny světelných podmínek a vrhání stínů. Jak potvrzují srovnávací studie zaměřené na reálné nasazení videodetekčních počítacích systémů, právě tyto faktory vnějšího prostředí představují největší výzvu pro udržení vysoké a stabilní provozní přesnosti [2].

V reálném nasazení bez existence předem připravených exaktních datových sad je standardní evaluační metodou vytvoření vlastní sady referenčních dat (tzv. ground truth) formou nezávislého manuálního sčítání z pořízeného videozáznamu [20]. Referenční hodnota je tak získána nezávislým pozorováním, čímž je stanoven skutečný počet průjezdů vozidel (N_{real}). Tento údaj je následně porovnán s počtem vozidel detekovaných automatizovaným systémem (N_{sys}) pomocí metriky poměrové přesnosti sčítání ($Accuracy_{count}$):

$$Accuracy_{count} = \left(1 - \frac{|N_{real} - N_{sys}|}{N_{real}}\right) \cdot 100\ [\%]$$

Tato metrika věrně a zcela exaktně reflektuje celkovou provozní úspěšnost systému při kontinuálním sčítání vozidel, a právě ona bude využita jako primární hodnoticí ukazatel v testovací části této práce.

1.7. Etické aspekty a GDPR

Při návrhu, nasazení a provozu jakýchkoliv kamerových systémů na veřejných prostranstvích a dopravních komunikacích je absolutně nezbytné zohlednit principy ochrany osobních údajů. Tyto povinnosti primárně vyplývají z platného Nařízení Evropského parlamentu a Rady (EU) 2016/679, obecně známého pod zkratkou GDPR (z angl. *General Data Protection Regulation*). Moderní kamerové systémy totiž ze své podstaty mohou neúmyslně zachytávat citlivé identifikační prvky, jakými jsou tváře chodců, řidičů nebo konkrétní registrační značky vozidel.

Navrhované řešení v rámci této bakalářské práce proto plně a striktně respektuje moderní přístup tzv. *Privacy by Design* (ochrana soukromí již od fáze samotného návrhu softwarové architektury). Tohoto principu je dosaženo právě díky integraci konceptu Edge AI. Veškeré algoritmické zpracování obrazových dat z kamery probíhá plně lokálně, přímo v operační paměti okrajového zařízení, které je umístěno bezprostředně u zdroje videa [12].

Díky lokální a okamžité inferenci se žádný surový obrazový signál, videozáznam ani snímky obsahující potenciální osobní údaje nikam neukládají, neodesílají se přes nezabezpečené sítě a nejsou nahrávány do žádného vzdáleného cloudového úložiště. Do databázového systému se

propisují a k následné vizualizaci odesílají pouze finální, přísně anonymizovaná a agregovaná numerická metadata – například celkový typ objektu (automobil, chodec), hrubý časový údaj, směr jeho pohybu a identifikátor sčítací brány.

Tento architektonický přístup bezezbytku naplňuje zásadu minimalizace osobních údajů, která je jedním z nosných a nejdůležitějších pilířů legislativy GDPR. Systém tak poskytuje plnohodnotná statistická data pro analýzu dopravy, aniž by jakýmkoliv způsobem zasahoval do soukromí účastníků provozu.

2. Návrh systému

Návrh monitorovacího systému vychází z analýzy provozních požadavků na nepřetržité zpracování obrazového toku v reálném čase přímo v místě instalace kamery. Při koncepčním návrhu byla zvolena metodika dekompozice systému do pěti funkčně nezávislých vrstev, které jsou vzájemně propojeny asynchronními komunikačními kanály. Tato kapitola postupně definuje funkční a nefunkční požadavky, představuje celkovou distribuovanou architekturu a zdůvodňuje volbu hardwarových prvků v čele s jednodeskovým počítačem Raspberry Pi 5 a akcelerátorem Hailo-8. Dále je detailně rozpracován objektový návrh aplikace pomocí diagramu tříd UML, logika prostorové filtrace vozidel v polygonální zóně ROI, mechanismus asynchronního přenosu zpráv protokolem MQTT a návrh relačního datového modelu v databázi PostgreSQL včetně strategie indexování a následné analytické vizualizace.

2.1. Požadavky na systém a volba technologií

Navrhovaný systém je určen pro automatizovanou analýzu silniční dopravy pomocí metod počítačového vidění. Cílem je detekce a sledování vozidel v reálném čase a následná extrakce statistických údajů o jejich průjezdech.

Mezi hlavní funkční požadavky patří:

- detekce vozidel v obrazové sekvenci,
- klasifikace detekovaných vozidel do kategorií,
- sledování klasifikovaných objektů napříč jednotlivými snímky,
- identifikace průjezdu klasifikovaných objektů definovanou oblastí zájmu (ROI),
- ukládání detekčních událostí do databáze,
- vizualizace agregovaných statistik.

Systém musí být schopen provozu v reálném čase na vestavěném zařízení s omezenými výpočetními prostředky. Z tohoto důvodu je navržena architektura založená na principu *edge computing*, kdy je inference neuronové sítě prováděna přímo na zařízení v místě sběru dat [14].

Volba softwarových technologií

Na základě definovaných funkčních a nefunkčních požadavků byl pro jednotlivé vrstvy navržené architektury zvolen technologický stack shrnutý v tabulce 2.1.

Komponenta systému	Zvolená technologie
Operační systém (Edge)	Raspberry Pi OS (Debian Linux) [16]
Programovací jazyk	Python 3.11 [21]
Zpracování videa	GStreamer [19]
AI inference a sledování	HailoRT + TAPPAS (YOLO) [22, 10]
Geometrická filtrace ROI	Shapely [23]
Komunikační protokol	MQTT (Paho-MQTT) [24]
Databázové úložiště	PostgreSQL + Psycopg [25]
Backend a REST API	FastAPI + Pandas / NumPy [26, 27, 28]
Vizualizační frontend	Streamlit + Plotly [29, 30]

Tabulka 2.1.: Přehled technologického stacku navrženého systému.

Volba uvedených technologií vychází z požadavku na vysokou propustnost zpracování obrazu při zachování nízké výpočetní a paměťové režie na okrajovém zařízení:

- **GStreamer a HailoRT (TAPPAS):** Na rozdíl od konvenčního zpracování obrazu v čistém jazyce Python (např. čtení snímků ve smyčce knihovny OpenCV [31]) umožňuje multimediální framework GStreamer [19] sestavit datovod z kompilovaných C/C++ zásuvných modulů. Dekódování síťového RTSP streamu, změna rozlišení, inference na akcelerátoru Hailo-8 pomocí knihoven TAPPAS [22] i sledování objektů tak probíhají s přímým předáváním paměťových bufferů bez zbytečného kopírování obrazových matic do paměti interpretu Pythonu.
- **Shapely a protokol MQTT:** Knihovna Shapely [23] poskytuje optimalizované planární geometrické operace v jazyce C (GEOS), které umožňují okamžité vyhodnocení přítomnosti těžiště vozidla v nepravidelném polygonu ROI. Lehký binární protokol MQTT [24] zajišťuje asynchronní přenos metadat s minimální síťovou režií a odděluje detekční uzel od databázového serveru.
- **PostgreSQL, FastAPI, Streamlit a Docker:** Relační databázový systém PostgreSQL s adaptérem Psycopg [25] byl zvolen pro svou nativní podporu časových typů s časovou zónou (`TIMESTAMPTZ`), geometrických datových typů (`POINT`) a efektivního dávkového vkládání. Backendový framework FastAPI [26] v kombinaci s knihovnami Pandas [27] a NumPy [28] poskytuje rychlé asynchronní REST API, na které navazuje webový dashboard v nástrojích Streamlit [29] a Plotly [30]. Platforma Docker [32] pak zaručuje plnou reprodukovatelnost a izolaci prostředí.

2.2. Výběr hardwaru

Pro zajištění spolehlivého chodu celého řešení je hardwarová část fyzicky rozdělena na prvek pro sběr obrazových dat a výpočetní uzel pro jejich zpracování.

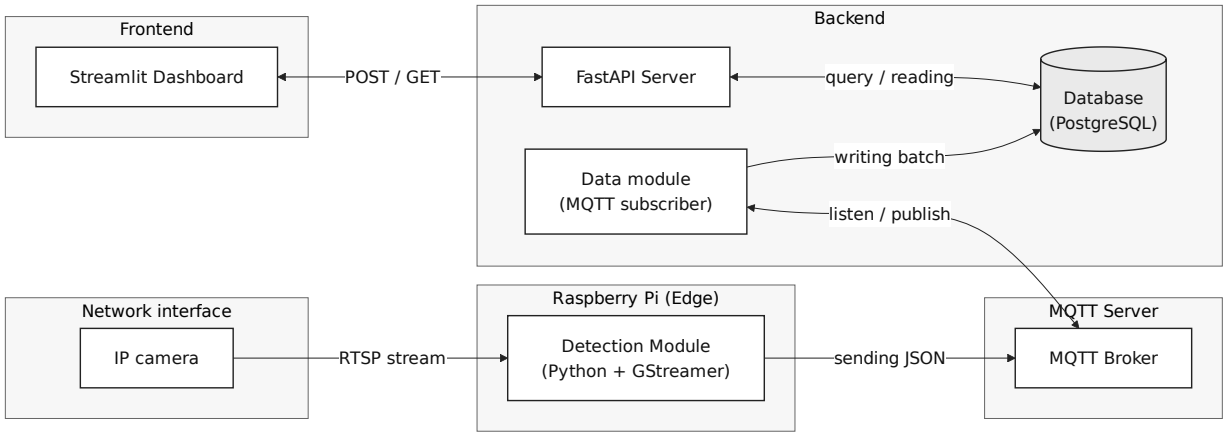
Kamerový systém: Pro dosažení dostatečné kvality detekce je využita IP kamera značky Hikvision, která umožňuje síťový přenos video streamu prostřednictvím protokolů RTSP nebo HTTP. Použití IP kamery dovoluje flexibilní umístění zařízení mimo samotný výpočetní uzel a současně poskytuje vyšší odolnost vůči povětrnostním podmínkám v porovnání s běžnými kamerovými moduly pro vestavěná zařízení.

Zpracovatelská jednotka: Jako výpočetní platforma na hraně sítě (edge zařízení), která přijímá datový tok z lokální sítě, je navrženo zařízení Raspberry Pi 5. Tato platforma představuje vhodný kompromis mezi výkonem, spotřebou a pořizovací cenou [16]. Pro zajištění real-time inference neuronové sítě je doplněna o dedikovaný AI akcelerátor Hailo-8 kompatibilní s touto platformou [17]. Tento přístup umožňuje efektivní zpracování obrazu přímo na místě bez nutnosti využití cloudových služeb.

Referenční výpočetní stanice (GPU): Pro účely kompilace modelů (která je paměťově i výpočetně náročná) a pro realizaci referenčního srovnávacího měření s nekvantizovaným modelem YOLO v plné přesnosti (FP32/FP16) byl zvolen přenosný počítač Acer Nitro V 15 (ANV15-51) vybavený dedikovanou grafickou kartou **NVIDIA GeForce RTX 2050** (architektura Ampere, 2 048 CUDA jader, 4 GB paměti GDDR6). Tato platforma slouží v experimentální části práce jako referenční srovnávací základna vůči nízkoenergetickému okrajovému uzlu s akcelerátorem Hailo-8.

2.3. Celková architektura a tok dat

Architektura systému je navržena jako vícevrstvá, distribuovaná a modulární. Jednotlivé komponenty jsou odděleny podle své funkce a komunikují spolu asynchronně prostřednictvím lehkého zprávového protokolu MQTT [24]. Toto rozdělení umožňuje lepší škálovatelnost, robustnost a minimalizaci vzájemné závislosti komponent, což plně odpovídá moderním architekturám dopravních monitorovacích systémů založených na počítačovém vidění [2].



Obrázek 2.1.: Navržená distribuovaná architektura systému a směr toku dat mezi vrstvami.

Na obrázku 2.1 je znázorněna celková topologie navrženého řešení. Systém je rozdělen do pěti funkčních vrstev, kterými postupně prochází tok dat:

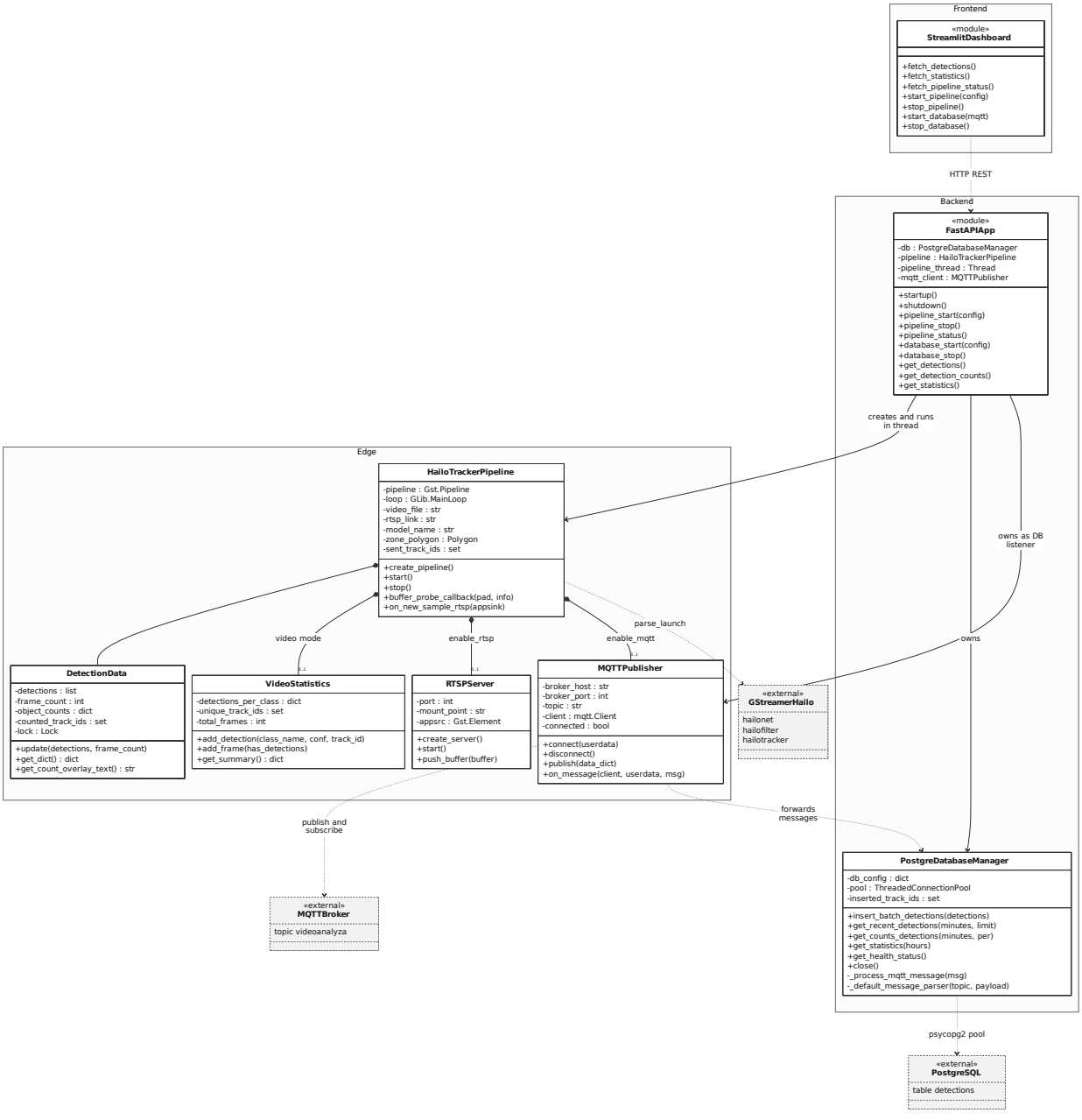
1. **Vrstva akvizice obrazu (Síťové rozhraní):** Koncová IP kamera poskytuje kontinuální video stream přenášený po lokální síti pomocí protokolu RTSP (*RTSP stream*).
2. **Vrstva zpracování videa (Edge vrstva):** Obrazová data jsou přijata na okrajovém zařízení Raspberry Pi 5 v řetězci GStreamer, předzpracována a předána detekčnímu modelu YOLO [3] na akcelerátoru Hailo-8. Následný sledovací algoritmus (na principu SORT [4]) přiřazuje objektům unikátní identifikátory a vyhodnocuje jejich průchod definovanou polygonální oblastí zájmu (ROI) [20]. Pro testovací a ladicí účely dokáže tento modul zpracovaný obraz s vykreslenými detekcemi souběžně vysílat jako výstupní RTSP stream nebo ukládat do souboru MP4.
3. **Komunikační vrstva (MQTT Server):** Vygenerované strukturované události o průjezdech vozidel jsou serializovány do formátu JSON a asynchronně odesílány (*sending JSON*) do centrálního MQTT brokeru [24].
4. **Datová a backendová vrstva (Backend):** Datový modul (MQTT subscriber) udržuje obousměrné spojení s brokerem (*listen / publish* pro odebírání zpráv a řízení odběru) a přijaté události dávkově zapisuje (*writing batch*) do normalizovaného schématu relační databáze PostgreSQL. Součástí backendu je aplikační server FastAPI, který s databází komunikuje obousměrně (*query / reading*) – odesílá analytické SQL dotazy a přijímá výsledné datové sady.
5. **Aplikační a vizualizační vrstva (Frontend):** Webový dashboard ve frameworku Streamlit komunikuje s FastAPI serverem obousměrně přes protokol HTTP (*POST / GET*) – pomocí požadavků `GET` načítá agregované dopravní statistiky a pomocí požadavků `POST` umožňuje vzdáleně spouštět či zastavovat detekční pipeline a databázové služby.

Oddělení jednotlivých kroků do samostatných vrstev umožňuje vysokou modularitu systému a minimalizaci vzájemné závislosti komponent.

2.4. Návrh softwarových modulů

Objektový návrh aplikace (UML)

Pro zajištění modularity, snadné testovatelnosti a budoucí rozšiřitelnosti byl softwarový kód systému navržen s využitím objektově orientovaného programování (OOP). Jednotlivé funkční celky od zpracování videa přes mezivrstvu pro přenos zpráv až po databázové operace a aplikační rozhraní, jsou zapouzdřeny do samostatných tříd a modulů (viz obrázek 2.2). Tento přístup umožňuje provozovat detekční část zcela autonomně z příkazové řádky, nebo ji dynamicky řídit prostřednictvím webového rozhraní.



Obrázek 2.2.: UML diagram tříd navrženého dopravního monitorovacího systému.

Struktura softwarové aplikace je rozdělena do následujících vrstev a klíčových tříd:

1. **Vrstva Edge (`Edge`):** Běží na okrajovém zařízení a zajišťuje akvizici obrazu, inferenci a odesílání událostí:
   - `HailoTrackerPipeline` – Ústřední řídicí třída detekčního modulu. Zapouzdřuje sestavení multimediálního řetězce (`Gst.Pipeline`), hlavní smyčku (`GLib.MainLoop`), komunikaci s externími prvky `GStreamerHailo` (`hailonet`, `hailofilter`, `hailotracker`) a registraci zpětného volání (`buffer_probe_callback`). Současně udržuje geometrickou definici oblasti zájmu (`zone_polygon`) a množinu již odeslaných identifikátorů (`sent_track_ids`).
   - `DetectionData` – Vláknově bezpečná (*thread-safe*) datová struktura využívající synchronizační zámek (`Lock`) pro uchovávání aktuálních detekcí, počtu snímků a generování překryvného textu (`get_count_overlay_text`).
   - `VideoStatistics` a `RTSPServer` – Volitelné kompozičně vázané třídy (s násobností `0..1`). `VideoStatistics` agreguje souhrnné metriky při offline analýze záznamů, zatímco `RTSPServer` umožňuje prostřednictvím prvku `appsrc` streamovat obraz s vykreslenými detekcemi po síti.
   - `MQTTPublisher` – Univerzální komunikační třída zapouzdřující klienta protokolu MQTT. Na straně edge pipeline publikuje serializované slovníky detekcí do externího uzlu `MQTTBroker` (téma `videoanalyza`), zatímco na straně serveru funguje jako odběratel předávající zprávy metodě `on_message`.
2. **Backendová vrstva (`Backend`):** Zajišťuje trvalé ukládání dat, správu databázových spojení a aplikační rozhraní:
   - `PostgreDatabaseManager` – Třída zodpovědná za perzistenci a čtení dat z externího databázového systému `PostgreSQL`. Využívá vláknový fond spojení (`ThreadedConnectionPool`), parsuje příchozí MQTT zprávy, provádí dávkové vkládání záznamů (`insert_batch_detections`) a poskytuje agregační metody pro analytické a stavové dotazy.
   - `FastAPIApp` – Aplikační modul poskytující REST API endpointy pro čtení statistik z databáze (`get_detections`, `get_statistics`) a zároveň umožňující vzdáleně spouštět a zastavovat detekční pipeline v samostatném vlákně (`Thread`).
3. **Prezentační vrstva (`Frontend`):**
   - `StreamlitDashboard` – Prezentační modul komunikující s modulem `FastAPIApp` přes protokol HTTP REST. Zajišťuje načítání dat (`fetch_detections`, `fetch_statistics`), interaktivní vykreslování grafů a ovládání chodu systému koncovým uživatelem.

Detekční modul a logika počítání (ROI)

Detekční modul je založen na konvoluční neuronové síti typu YOLO, která umožňuje rychlou lokalizaci a klasifikaci objektů v obraze [3]. Pro následné sledování těchto objektů napříč snímky je navržen algoritmus na principu SORT [4] (v rámci hardwarově optimalizovaného modulu `hailotracker`), případně DeepSORT [6] či ByteTrack. Každému sledovanému objektu je tak přiřazen unikátní identifikátor (ID), jenž je udržován po celou dobu jeho výskytu ve scéně.

Logika detekční zóny (ROI): Pro účely filtrace a počítání relevantních vozidel je v obraze definována virtuální oblast zájmu (ROI) ve tvaru obecného polygonu [20]. Polygonální reprezentace (implementovaná pomocí geometrické knihovny Shapely) umožňuje přesně vymezit sledovaný jízdní pruh na okružní křižovatce a odfiltrovat nežádoucí detekce mimo komunikaci. Započítání vozidla proběhne v okamžiku, kdy se geometrický střed jeho ohraničujícího rámečku (*bounding box*) ocitne uvnitř definovaného polygonu. Aby nedocházelo k vícenásobnému započítání téhož vozidla v po sobě jdoucích snímcích, systém udržuje v paměti množinu již zpracovaných identifikátorů `track_id` a každé unikátní vozidlo odešle ke zpracování pouze jednou.

Zpracování datového toku a asynchronního přenosu

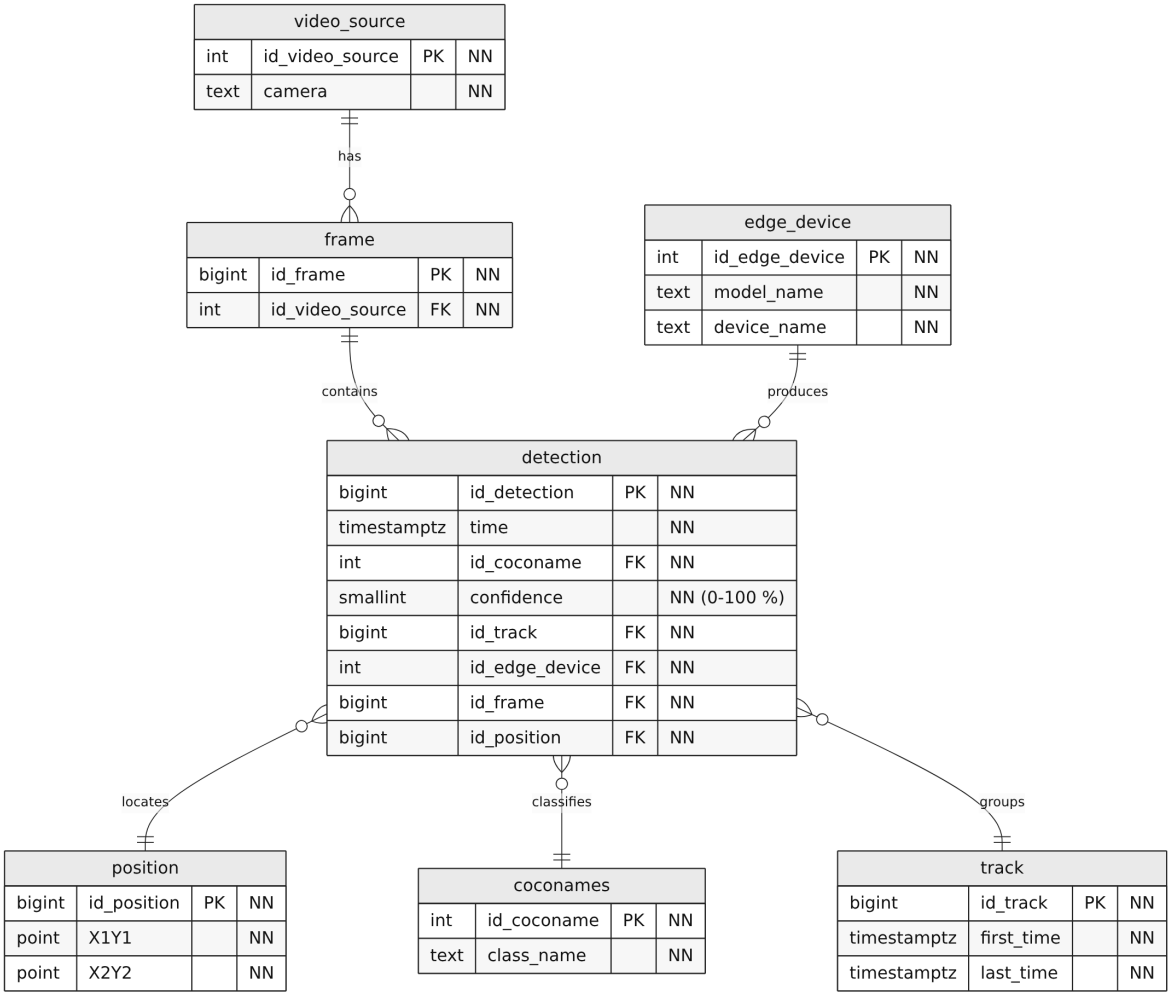
Aby okrajové zařízení zvládalo zpracovávat obrazový tok v reálném čase bez ztráty snímků, je vnitřní zpracování dat navrženo jako striktně oddělené (decoupled) na úrovni multimediálního zpracování i síťové komunikace:

1. **Zřetězené zpracování obrazu (GStreamer pipeline):** Dekódování síťového RTSP streamu, změna rozlišení na 640×640 pixelů, samotná inference na čipu Hailo-8 (ze zkompilovaného binárního souboru HEF) a přiřazování sledovacích identifikátorů jsou navrženy jako souvislý datovod v nativním C/C++ frameworku GStreamer. Řídicí vrstva v jazyce Python do obrazových matic přímo nezasahuje, ale pouze odebírá vygenerovaná textová metadata o detekcích prostřednictvím asynchronního zpětného volání (*buffer probe*).
2. **Asynchronní přenos metadat (MQTT návrh):** Přímý synchronní zápis do vzdálené databáze z vlákna zpracovávajícího video by v případě síťové odezvy blokoval výpočetní pipeline. Proto je přenos dat navržen na bázi modelu *Publish/Subscribe* pomocí protokolu MQTT [24]. Jakmile vozidlo projede polygonální zónou ROI, detekční modul sestaví lehkou datovou zprávu ve formátu JSON (obsahující identifikátor a název třídy, míru spolehlivosti *confidence*, souřadnice rámečku `bbox`, sledovací `track_id` a časové razítko `timestamp`) a neblokujícím způsobem ji publikuje na MQTT broker do tématu `/videoanalyza`.
3. **Dávková perzistence a obsluha dotazů (REST API):** Na straně serveru odebírá zprávy z MQTT brokeru nezávislá služba (subscriber). Pro minimalizaci režie databázových transakcí při hustém provozu je navrženo využití fondu spojení (*connection pooling*) a dávkového vkládání záznamů (*batch insert*). Nad databází stojí backendový server ve frameworku FastAPI, který prostřednictvím REST API endpointů poskytuje předagregovaná statistická

data prezentační vrstvě (Streamlit dashboardu) a současně umožňuje vzdálené řízení stavu detekční pipeline.

Návrh databázového úložiště a prezenční vrstvy

Strukturované detekční události přijaté z MQTT brokeru jsou trvale ukládány do relačního databázového systému PostgreSQL [25]. Pro zajištění datové integrity, eliminaci redundance (splnění třetí normální formy – 3NF) a efektivní provádění analytických dotazů byl navržen normalizovaný relační datový model. Namísto ukládání opakujících se textových řetězců do jediné ploché tabulky je schéma dekomponováno do sedmi entit propojených cizími klíči, jak znázorňuje entitně-relační (ER) diagram na obrázku 2.3.



Obrázek 2.3.: Entitně-relační (ER) diagram normalizovaného databázového schématu.

Navržené schéma se skládá z centrální tabulky detekčních událostí a šesti navazujících dimenzionálních a číselníkových tabulek:

- **`detection` (Centrální tabulka detekcí):** Uchovává záznamy o jednotlivých detekcích v čase. Primárním klíčem je atribut `id_detection` (`BIGINT`). Čas pořízení záznamu s přesností na časovou zónu udává atribut `time` (`TIMESTAMPTZ`). Míra spolehlivosti predikce

confidence je pro úsporu paměťového prostoru ukládána jako 2bajtové celé číslo (SMALLINT) vyjadřující procentuální jistotu modelu (0–100). Tabulka je prostřednictvím cizích klíčů (*Foreign Keys*) s kardinalitou $N : 1$ navázána na číselník tříd (id_coconame), sledovanou trajektorii (id_track), výpočetní uzel (id_edge_device), zdrojový snímek (id_frame) a prostorovou polohu (id_position).

- **coconames (Číselník detekovaných tříd):** Mapuje celočíselný identifikátor třídy id_coconame (INT) na textový název kategorie dle datasetu COCO v atributu class_name (TEXT, např. car, bus, truck, person). Vyčlenění názvů do samostatného číselníku výrazně zmenšuje velikost hlavní tabulky.
- **track (Evidence sledovaných trajektorií):** Reprezentuje unikátní fyzické objekty sledované v čase pod identifikátorem id_track (BIGINT). Atributy first_time a last_time (TIMESTAMPTZ) zaznamenávají přesný okamžik prvního a posledního zachycení daného vozidla ve scéně, což v analytické vrstvě umožňuje přímo vypočítat dobu zdržení vozidla v oblasti zájmu (*dwell time*).
- **position (Prostorová lokalizace objektu):** Uchovává geometrii ohraničujícího obdélníku (*bounding box*) pod primárním klíčem id_position (BIGINT). Namísto čtyř samostatných desetinných sloupců využívá nativní geometrický datový typ databáze PostgreSQL POINT – atribut X1Y1 definuje souřadnice levého horního rohu a X2Y2 souřadnice pravého dolního rohu detekovaného objektu.
- **frame a video_source (Evidence snímků a kamer):** Tabulka frame jednoznačně identifikuje zpracovaný obrazový snímek pomocí klíče id_frame (BIGINT) a odkazuje cizím klíčem id_video_source (INT) na tabulku video_source. Ta v atributu camera (TEXT) eviduje konkrétní zdroj videa (identifikátor IP kamery či cestu k záznamu).
- **edge_device (Číselník výpočetních uzlů):** Eviduje koncová zařízení pod klíčem id_edge_device (INT), název nasazeného detekčního modelu v atributu model_name (TEXT, např. yolov8m) a identifikátor výpočetní jednotky device_name (TEXT). Tato tabulka zajišťuje plnou připravenost systému na horizontální škálování do více dopravních lokalit a porovnávání různých verzí modelů.

Indexování: S ohledem na charakter analytických dotazů jsou nad cizími klíči a časovými atributy time (v tabulce detection) a first_time (v tabulce track) definovány B-tree indexy. Použití sestupného indexu nad časovým razítkem optimalizuje dotazy na nejnovější průjezdy a agregaci v časových oknech pro vizualizační dashboard

Návrh analytického rozhraní a vizualizace dat

Pro zpřístupnění uložených dopravních dat koncovému uživateli je nad databázovou vrstvou navrženo asynchronní REST API postavené na frameworku FastAPI [26], které odděluje datovou logiku od samotné prezentace. Numerické transformace, výpočet klíčových ukazatelů (celkový počet průjezdů, průměrná spolehlivost detekce, vytížení databáze) a časové agregace záznamů do

volitelných intervalů zajišťují knihovny NumPy [28] a Pandas [27]. Koncové webové rozhraní je navrženo ve frameworku Streamlit [29] v kombinaci s grafickou knihovnou Plotly [30] tak, aby poskytovalo jak přehledový analytický pohled (skládané sloupcové grafy časových řad, rozpad podle tříd vozidel a souhrnné metriky za zvolené období), tak ovládací panel (*System Control*) pro vzdálené spouštění a zastavování detekční pipeline a sledování stavu databázového připojení.

3. Implementace systému

Tato kapitola popisuje praktickou softwarovou realizaci navrženého dopravního monitorovacího systému v jazyce Python 3.11 [21]. Kompletní postupy instalace operačního systému, systémových závislostí, ovladačů a konfigurace automatického spouštění služeb jsou z důvodu rozsahu vyčleněny do externích příloh v elektronickém archivu práce (viz příloha A), kde jsou přehledně rozděleny na konfiguraci okrajového zařízení (Raspberry Pi 5 s akcelerátorem Hailo-8) a konfiguraci referenční stanice (PC s GPU).

První část této kapitoly se věnuje modularizaci zdrojového kódu, správě citlivých údajů pomocí proměnných prostředí a kontejnerizaci v prostředí Docker [32]. Následuje postup přípravy detekčního modelu YOLO – od exportu vah do meziformátu ONNX přes post-tréninkovou kvantizaci (PTQ) až po kompilaci do binárního souboru HEF pro akcelerátor Hailo-8. Stěžejní část kapitoly popisuje konstrukci multimediálního řetězce ve frameworku GStreamer [19], extrakci detekčních metadat ze snímkových bufferů, geometrické vyhodnocování průjezdů knihovnou Shapely [23] a implementaci asynchronního publikování událostí přes protokol MQTT [24] do databáze PostgreSQL [25] a webového rozhraní ve frameworcích FastAPI [26] a Streamlit [29].

3.1. Modularizace kódu, správa konfigurace a kontejnerizace

Pro zajištění udržitelnosti kódu a eliminaci duplicit mezi okrajovým zařízením (`rpi_src/`) a referenční pracovní stanicí (`pc_src/`) byla společná infrastruktura vyčleněna do sdíleného balíčku `common/`. Tento modul zapouzdřuje správu databázových spojení (`PostgreDatabaseManager`), komunikačního klienta (`MQTTPublisher`) a centralizované načítání konfigurace (`config.py`). Webová část na okrajovém zařízení byla rovněž rozdělena na samostatný backendový server (`API_with_streamlit.py`) a prezentační rozhraní (`streamlit_dashboard.py`).

Z hlediska kybernetické bezpečnosti jsou veškeré citlivé přihlašovací údaje (hesla k databázi PostgreSQL, přístupové údaje k MQTT brokeru a autentizační URL adresy RTSP kamerových toků) odděleny od zdrojového kódu a načítány dynamicky z proměnných prostředí prostřednictvím souboru `.env` (v repozitáři je verzována pouze anonymizovaná šablona `.env.example`).

Pro usnadnění replikovatelnosti experimentů a rychlé nasazení bez nutnosti manuální instalace závislostí (popsané v externí příloze A) byla implementována podpora kontejnerizace pomocí platformy Docker [32] a nástroje Docker Compose:

- **Kontejnerizované referenční prostředí (`pc_src/docker-compose.yml`):** Definuje kompletní stoh služeb zahrnující databázi PostgreSQL (s automatickou inicializací schématu pomocí skriptu `init_hailo.sql`) a výpočetní kontejner s akcelerací NVIDIA GPU pro běh referenčního detekčního skriptu.
- **Simulační prostředí (`rpi_src/docker/`):** Umožňuje otestovat průchod dat celým řetězcem (od simulovaného publikování MQTT událostí přes databázovou vrstvu až po vykreslení ve webovém dashboardu) v izolovaných kontejnerech i na zařízeních bez fyzicky připojeného akcelerátoru Hailo-8.
- **Automatizované testování a CI/CD pipeline (`tests/`, `.github/workflows/`):** Pro ověření spolehlivosti klíčových komponent byla pomocí frameworku `pytest` vytvořena sada jednotkových testů (`test_common.py`), která testuje načítání konfigurace, sestavování RTSP a databázových spojení a geometrickou logiku rozhodování o přítomnosti bodu uvnitř polygonu ROI. Tyto testy jsou spolu s validací syntaxe Docker Compose souborů a Kubernetes manifestů automaticky spouštěny při každé změně kódu v rámci CI/CD pipeline v prostředí GitHub Actions (`ci.yml`).

3.2. Příprava a kompilace detekčního modelu

Volba varianty modelu a předzpracování

V rámci implementace nebyl detekční model trénován od začátku, ale byl využit předtrénovaný model `YOLOv8m` (natrénovaný na referenčním datasetu COCO [8]), který byl zvolen i pro finální nasazení systému. Pro zprovoznění modelu na akcelerátoru Hailo-8 lze buď provést vlastní kompilaci do binárního formátu HEF (*Hailo Executable Format*) pomocí nástroje Hailo Dataflow Compiler (DFC) [33], nebo využít oficiálně předkompilované modely z repozitáře Hailo RPi5 Examples [34].

Během vývoje byly v rámci experimentů zkompilovány a otestovány vlastní varianty `YOLOv8n`, `YOLOv8s` a `YOLOv8m` (kompilační skripty a postup jsou zdokumentovány níže a přiloženy ve složce `rpi_src/model_compile/`). Při praktickém spuštění těchto vlastnoručně zkompilovaných souborů HEF v detekční pipeline se však projevila nekompatibilita výstupních post-processingových vrstev s cílovým prostředím, v jejímž důsledku inference neprobíhala na akcelerátoru Hailo-8, ale přesměrovala se na hlavní procesor (CPU) a dosahovala propustnosti pouze kolem 2 FPS. Pro finální nasazení a měření byla proto zvolena oficiálně předzpracovaná a plně hardwarově akcelerovaná varianta `yolov8m.hef` z knihovny `hailo-rpi5-examples` [34], která vykazovala optimální kompromis mezi rychlostí inference a přesností detekce vzdálenějších vozidel. Před samotnou inferencí jsou vstupní snímky vždy převzorkovány na rozlišení 640×640 pixelů a převedeny do barevného formátu RGB.

Export modelu do formátu ONNX

Prvním krokem přípravy pro kompilátor Hailo DFC je převod vah modelu z nativního formátu PyTorch (`.pt`) do standardizovaného meziformátu ONNX (*Open Neural Network Exchange*). Export byl proveden pomocí knihovny Ultralytics [10] s pevně definovanou velikostí vstupu a zjednodušením výpočetního grafu (ukázka 3.1).

Listing 3.1: Export modelu YOLOv8m do formátu ONNX.

```
from ultralytics import YOLO
model = YOLO("yolov8m.pt")
model.export(
    format="onnx",
    imgsz=640,
    opset=11,
    simplify=True
)
```

Výsledkem tohoto kroku je soubor `yolov8m.onnx`, který slouží jako přímý vstup pro optimalizační nástroje Hailo.

Kvantizace a kompilace do formátu HEF

Převod ONNX modelu do spustitelného formátu HEF byl proveden pomocí nástroje `hailomz` z knihovny Hailo Model Zoo [35]. Tento proces zahrnuje čtyři fáze:

1. překlad (parsing) ONNX grafu do meziformátu HAR (*Hailo Archive*),
2. optimalizaci a kvantizaci vah z 32bitové plovoucí řádové čárky (FP32) na 8bitová celá čísla (INT8),
3. alokaci hardwarových výpočetních zdrojů cílového čipu,
4. generování finálního binárního souboru HEF.

Pro správné provedení post-tréninkové kvantizace (PTQ) do formátu INT8 vyžaduje kompilátor kalibrační sadu čítající řádově stovky až tisíce snímků, na kterých analyzuje statistické rozložení aktivací v jednotlivých vrstvách. Pro tento účel byla využita validační část datasetu COCO (`val2017`) obsahující 5 000 snímků [8]. Vzhledem k vysoké výpočetní a paměťové náročnosti procesu kompilace (který trvá v řádu desítek minut) probíhal tento krok na externí pracovní stanici pomocí příkazu uvedeného v ukázce 3.2.

Listing 3.2: Kompilace modelu YOLOv8m pomocí nástroje Hailo Model Zoo.

```
hailomz compile yolov8m \
    --hw-arch hailo8 \
    --calib-path ./coco_val2017 \
    --classes 80
```

Přestože vlastní kompilace generovala validní soubory `.hef`, vzhledem k výše popsanému propadu výkonu při neúplném mapování grafu na akcelerátor (2 FPS při běhu na CPU) byl pro ostrý provoz na zařízení Raspberry Pi 5 nasazen referenční předkompilovaný soubor `yolov8m.hef` z oficiálního repozitáře [34], který plně využívá hardwarové zdroje čipu Hailo-8 při nízké latenci a spotřebě energie.

3.3. Implementace zpracování videa a detekce

Konstrukce GStreamer pipeline

Zpracování obrazového toku v reálném čase je postaveno na zřetězení zásuvných modulů frameworku GStreamer [19]. Pipeline se připojuje k IP kameře přes protokol RTSP, provádí hardwarově či softwarově akcelerované dekódování komprimovaného formátu H.264, škáluje obraz na rozlišení 640×640 pixelů a předává jej specializovaným prvkům Hailo (ukázka 3.3).

Listing 3.3: Zjednodušená konstrukce GStreamer pipeline.

```
pipeline_str = (
    "rtspsrc location=rtsp://CAMERA_IP latency=150 "
    "! rtph264depay ! h264parse ! avdec_h264 "
    "! videoconvert ! videoscale "
    "! video/x-raw,width=640,height=640,format=RGB "
    "! hailonet hef-path=yolov8m.hef batch-size=1 "
    "nms-score-threshold=0.6 nms-iou-threshold=0.5 "
    "! hailofilter function-name=yolov8m "
    "! hailotracker iou-thr=0.5 "
    "! videoconvert ! fakesink"
)
```

Samotnou inferenci na akcelerátoru zajišťuje prvek `hailonet`. Parametr `batch-size=1` garantuje, že každý přijatý snímek je zpracován okamžitě bez čekání na naplnění dávky více snímků, což minimalizuje latenci. Parametry `nms-score-threshold` a `nms-iou-threshold` konfigurují algoritmus *Non-Maximum Suppression* (NMS), jenž filtruje detekce s nízkou spolehlivostí a odstraňuje vícenásobné překrývající se rámečky nad stejným vozidlem. Následující prvek `hailofilter` provádí post-processing tenzorů a modul `hailotracker`, pracující na principu algoritmu SORT [4] (kombinujícího Kalmanův filtr s párováním pomocí metriky IoU), přiřazuje objektům sledovací identifikátory na základě překryvu (`iou-thr=0.5`) [22].

Pro účely ladění nebo paralelního ukládání záznamu lze pipeline na konci rozvětvit pomocí prvku `tee`, který duplikuje datový tok do více nezávislých větví a tím umožňuje paralelní zpracování výstupu z pipeline, např. pro současné streamování a ukládání videa (ukázka 3.4).

Listing 3.4: Větvení výstupu pipeline pomocí prvku tee.

```
pipeline_str += "! tee name=t "
pipeline_str += (
    "t. ! queue ! appsink name=rtsp_sink "
    "t. ! queue ! x264enc ! mp4mux ! filesink location=output.mp4"
)
```

Aby nedocházelo k zahlcení paměti a postupnému narůstání zpoždění při případném kolísání výpočetního výkonu, jsou mezi klíčové fáze pipeline vloženy vyrovnávací fronty (`queue`), které oddělují jednotlivé fáze zpracování do samostatných vláken, s aktivovaným režimem `leaky=downstream`. V případě přetížení fronta automaticky zahodí nejstarší nezpracované snímky a upřednostní aktuální obraz z kamery.

Extrakce metadat pomocí Buffer Probe

Aby nebylo nutné převádět celé obrazové snímky z GStreameru do paměti Pythonu, je pro čtení výsledků detekce využit mechanismus *buffer probe*. Na zdrojový výstup (`src` pad) prvku `hailotracker` je zaregistrována zpětná volací funkce (*callback*), jež se automaticky spustí při průchodu každého snímku (ukázka 3.5).

Listing 3.5: Registrace buffer probe na výstupu sledovacího modulu.

```
hailotracker = self.pipeline.get_by_name('hailotracker')
pad = hailotracker.get_static_pad('src')
pad.add_probe(Gst.PadProbeType.BUFFER, self.buffer_probe_callback)
```

Uvnitř callback funkce jsou pomocí Hailo API přímo z paměťového bufferu GStreameru extrahovány objekty typu `HAILO_DETECTION`, které nesou informaci o detekované třídě, míře spolehlivosti, souřadnicích ohraničujícího obdélníku a přiděleném identifikátoru sledování (ukázka 3.6).

Listing 3.6: Extrakce detekčních metadat z GStreamer bufferu.

```
buffer = info.get_buffer()
roi = get_roi_from_buffer(buffer)
hailo_detections = roi.get_objects_typed(HAILO_DETECTION)

for detection in hailo_detections:
    label = detection.get_label()
    confidence = detection.get_confidence()
```

Filtrace v polygonální zóně (ROI) a eliminace duplicit

Pro každé detekované vozidlo je nejprve vypočítán geometrický střed jeho ohraničujícího rámečku (c_x, c_y). Pomocí geometrické knihovny `Shapely` [23] je následně testováno, zda se tento bod nachází uvnitř uživatelsky definovaného polygonu oblasti zájmu (ukázka 3.7).

Listing 3.7: Filtrace detekcí v definované polygonální ROI zóně.

```
# Bounding box center determination
cx = bbox.xmin() + bbox.width() * 0.5
cy = bbox.ymin() + bbox.height() * 0.5

# Check if object is in zone
detection_point = Point(cx, cy)
if not self.zone_polygon.contains(detection_point):
    continue
```

Vzhledem k tomu, že vozidlo projíždějící zónou ROI je zachyceno na desítkách po sobě jdoucích snímků, je nezbytné zabránit jeho opakovanému započítání. Aplikace si proto udržuje množinu již odeslaných identifikátorů `sent_track_ids`. Pokud se dané `tracking_id` v množině již nachází, je detekce přeskočena (ukázka 3.8).

Listing 3.8: Prevence duplicitního odeslání téhož vozidla.

```
if tracking_id in self.sent_track_ids:
    continue

self._mark_sent(tracking_id)
```

Jelikož callback funkce GStreameru běží v samostatném vlákně, je aktualizace sdílených stavových proměnných aplikace chráněna synchronizačním zámkem (`Lock`), čímž je zajištěna vláknová bezpečnost (*thread-safety*):

Listing 3.9: Vláknově bezpečná aktualizace detekčních dat.

```
with self.lock:
    self.detections = detections
    self.frame_count = frame_count
    self.timestamp = datetime.now().isoformat()
```

Měření propustnosti (FPS)

Pro průběžné sledování výkonnosti pipeline je přímo v callback funkci implementován výpočet snímkové frekvence FPS podle vztahu:

$$\text{FPS} = \frac{N_{\text{frames}}}{t}$$

kde N_{frames} představuje celkový počet zpracovaných snímků od spuštění měření a t je uplynulý čas v sekundách (ukázka 3.10).

Listing 3.10: Průběžný výpočet FPS během běhu pipeline.

```
self.frame_count += 1
elapsed_time = time.time() - self.start_time
fps = self.frame_count / elapsed_time
```

3.4. Komunikace, ukládání a vizualizace dat

Publikace událostí přes MQTT

Nově zaznamenané průjezdy vozidel zónou ROI jsou zformátovány do podoby slovníku, který obsahuje identifikátor a název třídy, míru spolehlivosti, souřadnice rámečku, sledovací ID a časové razítko. Pokud seznam obsahuje nové události a spojení s MQTT [24] brokerem je aktivní, data jsou serializována do formátu JSON a publikována do příslušného tématu (ukázka 3.11).

Listing 3.11: Sestavení a publikace detekčních událostí přes MQTT.

```
detection_dict = {
    "class_id": class_id,
    "class_name": label,
    "confidence": float(confidence),
    "bbox": bbox_coords,
    "track_id": tracking_id,
    "timestamp": current_timestamp
}

detection_list.append(detection_dict)

if detection_list and self.mqtt_publisher.connected:
    self.mqtt_publisher.publish(detection_list)
```

Identifikátor `track_id` je označen jako zpracovaný až v okamžiku úspěšného předání zprávy komunikačnímu modulu, což zaručuje, že při případném krátkodobém výpadku spojení nedojde ke ztrátě detekce.

Dávkové ukládání do databáze PostgreSQL

Na serverové straně přijímá zprávy samostatný proces (MQTT subscriber). Pro zajištění vysoké propustnosti při komunikaci s databází PostgreSQL využívá aplikace mechanismus *connection poolingu* (opakované využívání otevřených databázových spojení). Samotný zápis dat do tabulky `detections` (jejíž schéma bylo definováno v kapitole 2.3) je realizován pomocí dávkového vkládání (`execute_batch`) z knihovny Psycopg, které umožňuje vložit až 100 záznamů v rámci jediné databázové transakce (ukázka 3.12).

Listing 3.12: Dávkové vkládání detekčních záznamů do databáze PostgreSQL.

```
query = """
INSERT INTO detections
(timestamp_, frame_id, class_name, confidence, x1, y1, x2, y2, track_id
    )
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
"""
execute_batch(cursor, query, data, page_size=100)
conn.commit()
```

Implementace vizualizačního dashboardu

Implementace aplikačního rozhraní

Zpřístupnění uložených záznamů z databáze PostgreSQL pro prezentační vrstvu zajišťuje backendový server (`API_with_streamlit.py`) prostřednictvím asynchronních REST API endpointů ve frameworku FastAPI [26] (ukázka 3.13). Kromě čtení posledních detekcí (`/api/detections`) a souhrnných statistik (`/api/statistics`) poskytuje server také řídicí endpointy pro vzdálené spouštění a zastavování detekční pipeline.

Listing 3.13: Implementace REST API endpointu ve frameworku FastAPI pro čtení detekcí.

```
@app.get("/api/detections", response_model=List[DetectionResponse])
async def get_detections(minutes: int = 5, limit: int = 100):
    if db is None:
        raise HTTPException(status_code=503, detail="Database not
            initialized")
    detections = db.get_recent_detections(minutes=minutes, limit=limit)
    return [DetectionResponse(**dict(d)) for d in detections]
```

Implementace vizualizačního dashboardu

Samotné uživatelské rozhraní (`streamlit_dashboard.py`) je vytvořeno ve frameworku Streamlit [29] a načítá data z backendu přes protokol HTTP. Přijatá JSON data jsou převedena do struktury `DataFrame` knihovny Pandas [27], kde probíhá konverze časových razítek na typ `datetime`.

Pro dynamickou analýzu vývoje dopravy v čase využívá aplikace metodu `resample()`, jež umožňuje uživateli měnit šířku časového okna agregace (např. 1 minuta, 10 minut, 1 hodina) jak globálně, tak pro jednotlivé kategorie vozidel (ukázka 3.14).

Listing 3.14: Časová agregace detekcí podle tříd vozidel v knihovně Pandas.

```
df_timeline = df.set_index("timestamp")
```

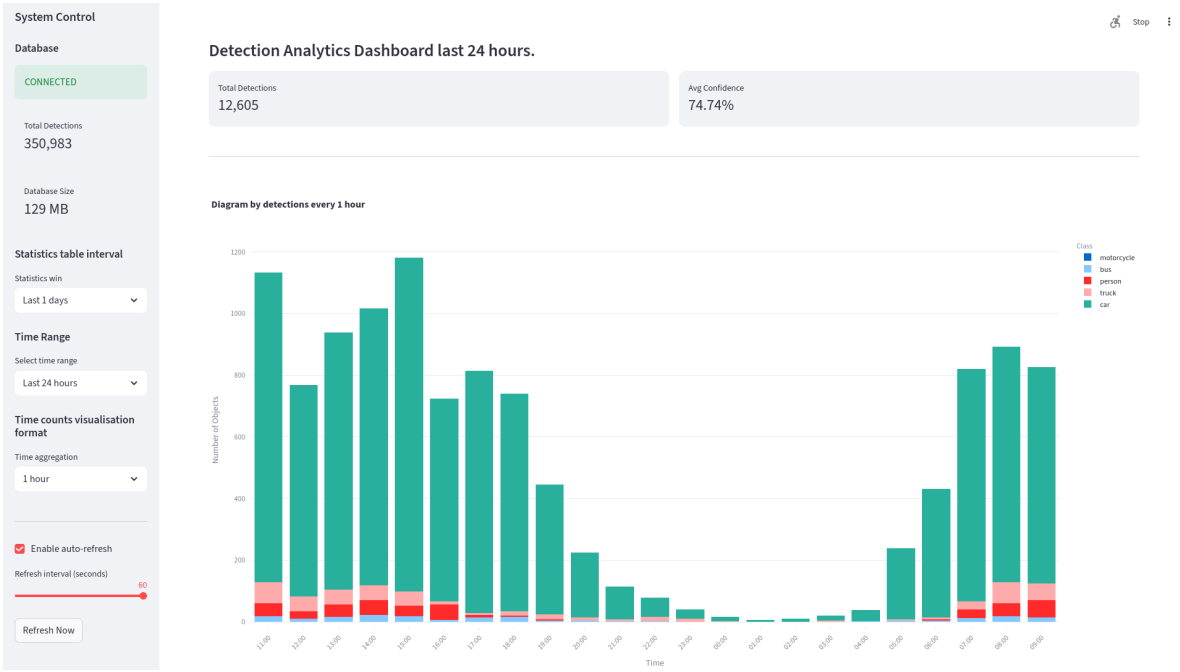
```
df_class_timeline = (
    df_timeline.groupby('class_name')
    .resample(selected_interval)
    .size()
    .reset_index(name="count")
)
```

Agregovaná časová řada je následně vykreslena pomocí knihovny Plotly [30] jako skládaný sloupcový graf (*stacked bar chart*), který přehledně zobrazuje celkovou intenzitu provozu i poměrné zastoupení jednotlivých typů vozidel v daných časových intervalech (ukázka 3.15).

Listing 3.15: Generování skládaného sloupcového grafu pomocí knihovny Plotly.

```
fig = px.bar(
    df_class_timeline,
    x="timestamp",
    y="count",
    color="class_name",
    barmode="stack"
)
```

Snímek obrazovky na obrázku 3.1 zachycuje výstup z dlouhodobého testovacího nasazení systému na reálném dopravním uzlu. V levém stavovém panelu (*System Control*) je patrné aktivní připojení k databázi PostgreSQL (`CONNECTED`), ve které bylo od spuštění sběru dat kumulativně uloženo již **350 983 detekčních záznamů**, přičemž díky normalizovanému databázovému schématu zabírá celé úložiště pouze **129 MB** diskového prostoru. V hlavní analytické části je zobrazen přehled za posledních 24 hodin s hodinovou agregací, během nichž systém zaznamenal **12 605 detekcí** s průměrnou mírou spolehlivosti (*Avg Confidence*) **74,74 %**. Ze skládaného sloupcového grafu je zřetelně patrný přirozený denní rytmus dopravního provozu – od minimálního nočního provozu mezi 00:00 a 04:00 přes ranní a dopolední dopravní špičku až po postupný večerní pokles intenzity, včetně barevného rozlišení zastoupení osobních automobilů (`car`), nákladních vozidel (`truck`), autobusů (`bus`), motocyklů (`motorcycle`) a chodců (`person`).



Obrázek 3.1.: Ukázka implementovaného vizualizačního dashboardu systému.

4. Testování a vyhodnocení

Zatímco v reálném nasazení na zařízení Raspberry Pi 5 pracuje navržený systém v nepřetržitém režimu, kde v reálném čase odesílá detekční události přes protokol MQTT do databáze a vizualizačního dashboardu, pro účely měření přesnosti sčítání a provozních vlastností bylo provedeno offline vyhodnocení nad zaznamenaným dopravním videem.

4.1. Testovací scénáře a metodické vymezení experimentu

Pro experimentální vyhodnocení byla použita přibližně čtyřminutová video sekvence zachycující reálnou dopravní situaci na okružní křižovatce za zhoršených zimních povětrnostních podmínek (husté sněžení, snížený kontrast a mokrá vozovka). Pro stanovení referenční hodnoty (*ground truth*, GT) byla všechna vozidla, která reálně projela vymezenou polygonovou oblastí zájmu (ROI), manuálně spočítána a klasifikována lidským pozorovatelem. Celkem bylo uvnitř měřené zóny zaznamenáno **112 reálných vozidel** (104 osobních automobilů, 3 autobusy a 5 dodávek či nákladních vozidel). V důsledku nepříznivého zimního počasí a sněhové pokrývky se na komunikaci během měření nevyskytovali žádní cyklisté (`bicycle`) ani motocyklisté (`motorcycle`). Vymezenou polygonovou zónou na vozovce rovněž neprocházel žádný chodec (`person`) – jediný chodec přítomný ve scéně se pohyboval po chodníku mimo aktivní polygon, a proto jsou v následujících tabulkách hodnoceny výhradně kategorie motorových vozidel (`Car`, `Bus`, `Truck`).

Aby bylo dosaženo metodicky korektního a přímého srovnání obou platforem, byla dopravní scéna analyzována s využitím totožné architektury neuronové sítě (`YOLOv8m`), shodného prostorového ořezu vstupního obrazu (640×640 px) a identické polygonální oblasti zájmu (ROI) na dvou hardwarových a softwarových konfiguracích:

- **Edge řešení (Raspberry Pi 5 + Hailo-8):** Detekce probíhala na cílovém okrajovém zařízení s AI akcelerátorem Hailo-8 (26 TOPS), post-tréninkově kvantizovaným modelem `YOLOv8m` v 8bitové celočíselné aritmetice (INT8, binární formát HEF) a sledovacím modulem `hailotracker` v rámci nativní GStreamer pipeline.
- **Referenční GPU řešení (PC + NVIDIA GeForce RTX 2050):** Detekce byla realizována na přenosném počítači Acer Nitro V 15 vybaveném dedikovanou grafickou kartou s podporou CUDA v nekvantizované přesnosti (FP32/FP16) s využitím knihovny Ultralytics, téhož modelu `YOLOv8m` (`yolov8m.pt`) a sledovacího algoritmu ByteTrack [7] (popsaného v kapitole 1.1), který byl zvolen z důvodu vysoké rychlosti zpracování v reálném čase (vysoké FPS) a odolnosti vůči zákrytům.

Sjednocení parametrů a metodická specifika srovnání: Při návrhu srovnávacího experimentu byly sjednoceny klíčové hyperparametry detekce i prostorové filtrace, přičemž u výsledků je třeba zohlednit tři architektonická specifika obou prostředí:

1. *Kvantizace vah (INT8 vs. FP32/FP16) a nastavení prahů:* Obě platformy využívají stejnou síť `YOLOv8m` se sjednoceným prahem spolehlivosti pro zakládání stop ($0{,}60$) a prahem překryvu NMS (IoU $= 0{,}50$). Rozdíl ve výsledcích tak přímo odráží vliv 8bitové kvantizace na čipu Hailo-8 oproti plné přesnosti na GPU v kombinaci se sledovacím algoritmem (`hailotracker` vs. dvoufázové párování ByteTrack s pamětí `track_buffer` = 50 snímků).
2. *Identická polygonální geometrie ROI:* Na obou platformách byla pomocí knihovny Shapely aplikována shodná pětiúhelníková oblast zájmu definovaná normalizovanými vrcholy $[(0{,}10; 0{,}85), (0{,}46; 0{,}85), (0{,}60; 0{,}90), (0{,}45; 0{,}90), (0{,}40; 1{,}00)]$. Polygon přesně kopíruje perspektivu měřeného jízdního pruhu na výjezdu z okružní křižovatky a eliminuje falešné detekce mimo sledovanou komunikaci (včetně chodce pohybujícího se mimo tuto zónu).
3. *Správa vyrovnávacích front při čtení souboru (4 957 vs. 5 113 snímků):* Oběma systémům byl předložen totožný zdrojový video soubor o celkovém počtu 5 113 snímků (odpovídající cca 255,6 s při 20 FPS). Zatímco referenční GPU implementace využívající bezztrátovou blokující frontu (`queue.Queue`) zpracovala všech 5 113 snímků, nativní GStreamer pipeline na Raspberry Pi 5 analyzovala 4 957 snímků. Rozdíl 156 snímků ($\sim 3{,}05\ \%$ délky záznamu) je způsoben prvkem `queue leaky=downstream max-size-buffers=5` před modulem `hailofilter`, který je v architektuře GStreameru navržen pro živé RTSP streamování a při nárazovém čtení z lokálního souboru (`filesrc`) zahazuje mezisnímky při zaplnění fronty.

4.2. Vyhodnocení přesnosti detekce a matice záměn

Pro posouzení kvality detekce byly z vyexportovaných telemetrických souborů ve složce `results/` (pro okrajový akcelerátor Hailo-8 a referenční GPU) analyzovány celkové počty hrubých snímkových detekcí uvnitř zóny ROI, průměrné míry spolehlivosti (*average confidence*) a složky matice záměn vůči manuálně stanovenému počtu 112 objektů.

Z hlediska míry spolehlivosti (*confidence score*) jednotlivých snímkových predikcí dosáhly obě platformy s modelem `YOLOv8m` velmi vyrovnaných výsledků, jak dokládá tabulka 4.1. Vážený průměr spolehlivosti napříč všemi detekovanými snímky činil 0,7730 u akcelerátoru Hailo-8 (nevážený průměr detekovaných tříd vozidel 0,7415) a 0,7414 u referenčního GPU řešení (nevážený průměr tříd vozidel 0,7307). Mírně nižší vážený průměr a nižší počet hrubých snímkových detekcí na GPU (3 732 oproti 4 780 na Hailo-8) souvisí s druhou fází algoritmu ByteTrack, která při zákrytech v hustém sněžení udržuje existující stopy i pomocí detekcí s nižším skóre spolehlivosti, zatímco u kvantizovaného modelu na Hailo-8 docházelo při rozpadu stopy k opakovanému generování nových detekčních záznamů.

Třída	Snímkové det. (Hailo)	Prům. conf. (Hailo)	Snímkové det. (GPU)	Prům. conf. (GPU)
Car	4 349	0,7787	3 283	0,7485
Bus	127	0,7400	111	0,7860
Truck	304	0,7057	338	0,6577
Celkem	**4 780**	**0,7415 (váž. 0,7730)**	**3 732**	**0,7307 (váž. 0,7414)**

Tabulka 4.1.: Přehled celkového počtu snímkových detekcí v zóně ROI a průměrné míry spolehlivosti (*confidence*) modelu `YOLOv8m` dle JSON logů.

Vyhodnocení matice záměn na úrovni unikátních objektů: Vzhledem k tomu, že testovací video nedisponuje snímkovou (*frame-by-frame*) anotací souřadnic všech ohraničujících obdélníků, nebylo možné provést automatizované prostorové párování jednotlivých trajektorií pomocí metriky IoU a Maďarského algoritmu (jaké využívají standardy MOTA či IDF1[1]). Matice záměn v tabulce 4.2 proto kombinuje vizuální kontrolu průjezdů na referenčním GPU záznamu s makroskopickým vyhodnocením koncových počtů unikátních stop:

- **Referenční GPU řešení ($N_{\text{GPU}} = 111$):** Manuální kontrolou záznamu oproti 112 skutečným průjezdům bylo zjištěno celkem $FN_{\text{GPU}} = 5$ chybějících detekcí (3 osobní automobily a 2 dodávky, které v hustém sněžení či zákrytu nepřekročily práh pro inicializaci stopy) a $FP_{\text{GPU}} = 4$ falešně pozitivní detekce (2 u osobních automobilů, 1 u dodávky a 1 u autobusu). Počet správně zachycených vozidel tak činí $TP_{\text{GPU}} = 112 - 5 = 107$ a celkový počet vygenerovaných záznamů je $TP + FP = 107 + 4 = 111$. (V surovém výpisu JSON logu figuruje rozpad 101 `Car`, 4 `Bus` a 6 `Truck`, neboť 2 větší osobní vozy byly modelem zařazeny do třídy `Truck`).
- **Edge řešení s akcelerátorem Hailo-8 ($N_{\text{Hailo}} = 127$):** Kvantizovaný model zachytil uvnitř polygonu všechna projíždějící osobní vozidla i autobusy ($FN = 0$ u tříd `Car` a `Bus`), avšak z 5 dodávek (`Truck`) klasifikoval do této třídy pouze 3 vozidla ($TP_{\text{Truck}} = 3$, $FN_{\text{Truck}} = 2$, zbylé 2 dodávky přešly do třídy `Car`). Vzhledem k vyšší míře fragmentace stop (*ghostingu*) za sloupem veřejného osvětlení, kterou nelze u rychle se měnících ID manuálně separovat, jsou přebytečné detekce na Hailo-8 stanoveny makroskopicky jako $FP = \max(0, N_{\text{sys}} - GT)$, což odpovídá $FP_{\text{Car}} = 120 - 104 = 16$ a $FP_{\text{Bus}} = 4 - 3 = 1$ (celkem $TP_{\text{Hailo}} = 110$, $FP_{\text{Hailo}} = 17$, $FN_{\text{Hailo}} = 2$).

[1] **MOTA** (*Multiple Object Tracking Accuracy*) je souhrnná metrika přesnosti víceobjektového sledování kombinující v každém snímku chyby typu FP, FN a počet nechtěných změn identit (*ID switches*). **IDF1** (*Identification F1 Score*) vyjadřuje harmonický průměr přesnosti a úplnosti zachování identity sledovaného objektu po celou dobu jeho trajektorie [5].

		Edge řešení (Hailo-8)			Referenční GPU řešení		
Třída objektu	**Ruční (GT)**	**TP**	**FP**	**FN**	**TP**	**FP**	**FN**
Car	104	104	16	0	101	2	3
Bus	3	3	1	0	3	1	0
Truck	5	3	0	2	3	1	2
Celkem	**112**	**110**	**17**	**2**	**107**	**4**	**5**

Tabulka 4.2.: Matice záměn (TP, FP, FN) pro unikátní sledované objekty v zóně ROI s modelem `YOLOv8m` vůči referenčnímu sčítání (GT $= 112$).

Z agregovaných složek matice záměn ($TP_{\text{Hailo}} = 110$, $FP_{\text{Hailo}} = 17$, $FN_{\text{Hailo}} = 2$ pro 127 detekcí Hailo-8 a $TP_{\text{GPU}} = 107$, $FP_{\text{GPU}} = 4$, $FN_{\text{GPU}} = 5$ pro 111 detekcí GPU) byly vypočítány celkové detekční metriky:

- **Precision (Přesnost):** Precision $= \frac{TP}{TP+FP}$
  - Edge řešení (Hailo-8): $\frac{110}{110+17} = \frac{110}{127} = \mathbf{86{,}61}\ \%$
  - Referenční GPU řešení: $\frac{107}{107+4} = \frac{107}{111} = \mathbf{96{,}40}\ \%$
- **Recall (Úplnost):** Recall $= \frac{TP}{TP+FN}$
  - Edge řešení (Hailo-8): $\frac{110}{110+2} = \frac{110}{112} = \mathbf{98{,}21}\ \%$
  - Referenční GPU řešení: $\frac{107}{107+5} = \frac{107}{112} = \mathbf{95{,}54}\ \%$
- **F1-skóre (Harmonický průměr Precision a Recall):** $F1 = \frac{2 \cdot TP}{2 \cdot TP+FP+FN}$
  - Edge řešení (Hailo-8): $\frac{220}{220+17+2} = \frac{220}{239} = \mathbf{92{,}05}\ \%$
  - Referenční GPU řešení: $\frac{214}{214+4+5} = \frac{214}{223} = \mathbf{95{,}96}\ \%$
- **Detekční Accuracy (Critical Success Index při $TN = 0$):** Accuracy $= \frac{TP}{TP+FP+FN}$
  - Edge řešení (Hailo-8): $\frac{110}{110+17+2} = \frac{110}{129} = \mathbf{85{,}27}\ \%$
  - Referenční GPU řešení: $\frac{107}{107+4+5} = \frac{107}{116} = \mathbf{92{,}24}\ \%$

4.3. Vyhodnocení přesnosti sčítání a klasifikace

Navazujícím krokem bylo vyhodnocení celkové provozní přesnosti sčítání dopravy pomocí poměrové metriky $\text{Accuracy}_{\text{count}}$, definované v teoretické části práce vztahem:

$$\text{Accuracy}_{\text{count}} = \left(1 - \frac{|N_{\text{real}} - N_{\text{sys}}|}{N_{\text{real}}}\right) \cdot 100\ [\%]$$

kde N_{real} představuje skutečný počet objektů, které projely zónou ROI dle manuální anotace (GT $= 112$), a N_{sys} celkový počet unikátních objektů (`unique_objects_tracked`) zaznamena-

ných systémem ve výstupních JSON souborech. Porovnání výsledků pro jednotlivé třídy i celkový součet uvádí tabulka 4.3.

Třída objektu	**Ruční (GT)**	**Hailo-8 (Edge)**	**GPU (Ref.)**	**Odchylka Hailo**	**Odchylka GPU**
Car (osobní vůz)	104	120	101	$+16$	-3
Bus (autobus)	3	4	4	$+1$	$+1$
Truck (nákladní vůz)	5	3	6	-2	$+1$
Celkem (N)	**112**	**127**	**111**	**$+15$**	**-1**
Accuracy$_{\text{count}}$	**100,00 %**	**86,61 %**	**99,11 %**	–	–

Tabulka 4.3.: Porovnání počtu započítaných unikátních objektů s referenční hodnotou (ground truth) při sjednocené konfiguraci modelu YOLOv8m.

Při interpretaci výsledků v tabulce 4.3 a tabulce 4.2 vystupují do popředí tři klíčové poznatky:

1. **Celková přesnost sčítání (Accuracy$_{\text{count}}$ vs. F1-skóre):** V celkovém součtu unikátních objektů dosáhlo referenční GPU řešení poměrové přesnosti sčítání **99,11 %** (111 detekovaných vozidel oproti 112 skutečným, odchylka -1 vozidlo), přičemž této těsné shody v celkovém součtu bylo částečně dosaženo vzájemnou kompenzací 5 falešně negativních a 4 falešně pozitivních detekcí (skutečné F1-skóre činí **95,96 %**). Edge řešení s akcelerátorem Hailo-8 dosáhlo celkové poměrové přesnosti sčítání **86,61 %** (127 detekovaných vozidel, odchylka $+15$ vozidel) a F1-skóre **92,05 %** (rozdíl v F1-skóre mezi oběma platformami tak činí pouhých 3,91 procentního bodu, přičemž v úplnosti záchytu *Recall* dosáhlo Hailo-8 dokonce vyšší hodnoty 98,21 % oproti 95,54 % na GPU).
2. **Chování na dominantní třídě `Car` (*Ghosting* vs. vyvážený tracking):** U nejpočetnější kategorie osobních automobilů (`Car`, GT $= 104$) zaznamenalo GPU řešení 101 vozidel (Accuracy$_{\text{count(Car)}} = 97{,}12\ \%$, odchylka -3, při $FN = 3$ a $FP = 2$), zatímco řešení Hailo-8 napočítalo 120 vozidel (Accuracy$_{\text{count(Car)}} = 84{,}62\ \%$, odchylka $+16$). Na straně GPU dokázal dvoufázový algoritmus ByteTrack udržet kontinuitu stop i při průjezdu hustým sněžením s minimem falešných duplicit. Naopak u čipu Hailo-8 vedlo drobné kolísání rozměrů kvantizovaných INT8 ohraničujících rámečků při zákrytu za sloupem veřejného osvětlení k dočasnému poklesu překryvu IoU v modulu `hailotracker` a následnému přidělení nového `track_id` témuž vozidlu (tzv. *ghosting*).
3. **Klasifikace tříd `Bus`, `Truck` a filtrace mimo ROI:** V kategorii autobusů (`Bus`, GT $= 3$) dosáhly obě platformy identického výsledku 4 detekovaných vozidel (1 falešně pozitivní detekce na každé platformě, $FP = 1$). V kategorii dodávek a nákladních vozidel (`Truck`, GT $= 5$) měly obě platformy 2 falešně negativní detekce ($FN = 2$, kdy byly menší dodávky v sněžení zařazeny pod osobní automobily nebo nepřekročily práh); na GPU se navíc v surovém JSON výstupu objevilo 6 záznamů `Truck` v důsledku 1 falešně pozitivní detekce dodávky a přeřazení 2 větších osobních vozů do této třídy. Chodec pohybující se mimo

vymezený polygon ROI nebyl ani na jedné platformě započítán (0 detekcí), což potvrzuje 100% funkčnost geometrické filtrace.

4.4. Vyhodnocení výkonnosti systému

Provozní parametry obou implementací byly vyhodnoceny na základě telemetrických logů vygenerovaných po dokončení analýzy záznamů (tabulka 4.4). Díky odstranění umělého časového brzdění (*throttlingu*) u obou souborových vstupů a nasazení stejného modelu `YOLOv8m` o vstupním rozlišení 640×640 px odrážejí naměřené hodnoty reálnou výpočetní propustnost obou softwarových řetězců:

- **Na Edge platformě (Raspberry Pi 5 + Hailo-8)** zpracovala nativní C/C++ GStreamer pipeline 4 957 snímků za 168,65 s, čímž dosáhla průměrné propustnosti **29,39 FPS** (průměrná doba zpracování snímku **34,02 ms**).
- **Na referenční GPU platformě (NVIDIA GeForce RTX 2050)** zpracovala asynchronní Python pipeline všech 5 113 snímků za 199,43 s, což odpovídá průměrné propustnosti **25,64 FPS** (průměrná doba zpracování snímku **39,00 ms**).

Měřený parametr	**Edge (RPi 5 + Hailo-8)**	**Referenční GPU (RTX 2050)**
Použitý detekční model	`YOLOv8m` (INT8, HEF)	`YOLOv8m` (FP32/FP16, PT)
Rozlišení vstupního tenzoru	640×640 px	640×640 px
Sledovací algoritmus a ROI	`hailotracker` + Polygon	ByteTrack + Polygon
Celkový počet zpracovaných snímků	4 957	5 113
Snímky s detekcí v ROI	3 594 (72,5 %)	3 205 (62,7 %)
Celková doba běhu (`processing_time`)	168,65 s	199,43 s
Průměrná propustnost (FPS)	**29,39 FPS**	**25,64 FPS**
Průměrná doba zpracování snímku (FPS^{-1})	**∼34,02 ms**	**∼39,00 ms**
Využití CPU (Raspberry Pi 5)	25–35 %	–
Typický příkon celé sestavy	∼8–12 W	$>$ 100 W

Tabulka 4.4.: Přehled provozních parametrů obou testovacích běhů s modelem `YOLOv8m` dle telemetrických JSON logů.

Analýza propustnosti a rozlišení od koncové (*end-to-end*) latence: Z naměřených dat vyplývá pozoruhodný výsledek: nízkoenergetický okrajový uzel Raspberry Pi 5 s akcelerátorem Hailo-8 dosáhl vyšší celkové propustnosti (29,39 FPS, krok pipeline 34,02 ms) než dedikovaná grafická karta v přenosném počítači (25,64 FPS, krok pipeline 39,00 ms). Příčinou je architektonický rozdíl v obsluze multimediálního toku. Zatímco na platformě Hailo-8 probíhá dekódování, ořez, škálování, 8bitová inference i H.264 enkódování ve zřetězených C/C++ vláknech frameworku GStreamer

s přímým předáváním paměťových bufferů, v referenční Python pipeline na PC vzniká režie při převodu snímků z knihovny PyAV do polí NumPy, vykreslování v OpenCV a synchronním zápisu výstupního MP4 souboru přes `cv2.VideoWriter`. Obě platformy nicméně s výraznou rezervou překonávají stanovený požadavek na propustnost v reálném čase (definovaný v kapitole 1.5 jako minimálně 20 FPS, tj. průměrná doba zpracování snímku pod 50 ms).

Z architektonického hlediska je však nutné od této převrácené hodnoty propustnosti ($\text{FPS}^{-1} = 34{,}02$ ms) striktně odlišit celkovou *end-to-end* latenci živého přenosu (zpoždění mezi fyzickým dějem před objektivem IP kamery a vygenerováním MQTT události). Při živém RTSP streamování obsahuje prvek `rtspsrc` síťovou vyrovnávací paměť nastavenou na `latency=150` (150 ms) pro kompenzaci kolísání sítě (*jitteru*), takže celková přenosová a výpočetní latence živého kanálu se pohybuje kolem 185–200 ms, přičemž díky režimu `leaky=downstream` a parametru `batch-size=1` tato hodnota v čase dále nenarůstá.

Vliv AI akcelerátoru a GStreameru na vytížení CPU: Zásadním přínosem integrace modulu Hailo-8 (26 TOPS) přes sběrnici PCIe je téměř úplné odlehčení hlavnímu procesoru ARM Cortex-A76 na desce Raspberry Pi 5. Zatímco samotný výpočet konvolučních vrstev modelu `YOLOv8m` pouze na CPU by vedl k 100% vytížení všech jader a propadu snímkové frekvence na řádově 1 až 2 FPS, při využití akcelerátoru Hailo-8 a nativních C/C++ pluginů frameworku GStreamer se průměrné vytížení CPU pohybuje pouze v rozmezí 25 až 35 %. Procesor tak obsluhuje pouze dekódování síťového RTSP streamu, geometrickou filtraci v knihovně Shapely a asynchronní odesílání JSON zpráv přes MQTT, což zajišťuje nízké zahřívání a dlouhodobou provozní stabilitu při zlomkové spotřebě elektrické energie ($\sim$8–12 W).

5. Diskuze a prezentace výsledků

Experimentální vyhodnocení ověřilo funkčnost navrženého systému na nízkoenergetické okrajové platformě (Edge AI) a díky sjednocení detekční architektury (`YOLOv8m`), prahových hodnot i polygonální oblasti zájmu (ROI) umožnilo přímou a objektivní konfrontaci s referenčním řešením na grafické kartě (GPU).

Z hlediska sčítání dopravního proudu v náročných zimních podmínkách (husté sněžení, snížený kontrast) vůči manuálnímu referenčnímu sčítání uvnitř polygonu ROI (112 vozidel) ukázaly výsledky jasné rozdělení silných stránek obou platforem. Referenční GPU řešení s modelem v plné přesnosti (FP32/FP16) a algoritmem ByteTrack dosáhlo lepších výsledků ve většině kvalitativních a statistických metrik: zaznamenalo 111 objektů, čímž v celkovém součtu dosáhlo poměrové přesnosti sčítání $\text{Accuracy}_{\text{count}} = 99{,}11\ \%$, vyšší přesnosti detekce $\text{Precision} = 96{,}40\ \%$, vyššího F1-skóre $95{,}96\ \%$ i indexu kritické úspěšnosti $\text{CSI} = 92{,}24\ \%$ (při $\text{Recall} = 95{,}54\ \%$ vycházejícím ze 107 správných, 4 falešně pozitivních a 5 falešně negativních detekcí). Navržené Edge řešení s akcelerátorem Hailo-8 a 8bitově kvantizovaným modelem (INT8) naproti tomu překonalo GPU v úplnosti záchytu $\text{Recall} = 98{,}21\ \%$ (zachytilo 110 ze 112 vozidel) a v průměrné míře spolehlivosti snímkových predikcí (vážený průměr 0,7730 oproti 0,7414), avšak vlivem vícenásobného započítání některých vozidel za překážkou zaznamenalo celkem 127 objektů ($\text{Accuracy}_{\text{count}} = 86{,}61\ \%$, $\text{Precision} = 86{,}61\ \%$ a $F1 = 92{,}05\ \%$). Detailní rozbor výsledků přinesl tři klíčové poznatky:

- **Kompromis mezi kvantizací (INT8) a plnou přesností (FP32/FP16):** Porovnání totožného modelu `YOLOv8m` na obou platformách ukázalo, že obě varianty vykazují odlišný charakter chyb. Na GPU v plné numerické přesnosti dokázal dvoufázový algoritmus ByteTrack velmi účinně potlačit falešné štěpení stop ($FP = 4$ napříč celým záznamem), avšak 3 osobní automobily a 2 dodávky v hustém sněžení nepřekročily práh detekce ($FN = 5$). Vzájemná kompenzace těchto chyb (-5 přehlédnutých a $+4$ falešně pozitivní) vedla k téměř shodnému celkovému součtu 111 vozidel oproti 112 skutečným. Na akcelerátoru Hailo-8 model sice nepřehlédl žádný osobní automobil ani autobus ($FN = 0$), avšak při zákrytech vozidel za sloupem veřejného osvětlení docházelo k mírnému kolísání rozměrů kvantizovaných INT8 ohraničujících rámečků, což vede k občasné fragmentaci stopy (*ghostingu*) v modulu `hailotracker` a nadhodnocení počtu osobních automobilů (120 oproti 104, tj. $+16$).
- **Přínos sjednocené polygonální ROI zóny:** Nasazení identického pětiúhelníkového polygonu v knihovně Shapely na obou platformách odstranilo dřívější anomálie spojené se změnou třídy vozidla během dlouhého průjezdu širokým obdélníkem (*class switching*). Po-

lygon přesně vymezil měřený profil vozovky na výjezdu z okružní křižovatky, což vedlo ke shodnému počtu detekovaných autobusů (4 na obou platformách oproti GT $= 3$, tedy $FP = 1$) i ke spolehlivému odfiltrování chodce pohybujícího se po chodníku mimo měřenou zónu.

- **Vliv architektury datovodu na propustnost (29,39 FPS vs. 25,64 FPS) a metodické limity:** Okrajové zařízení Raspberry Pi 5 s čipem Hailo-8 dosáhlo vyšší snímkové propustnosti (29,39 FPS, 34,02 ms/snímek) než referenční GPU pipeline (25,64 FPS, 39,00 ms/snímek), což je dáno využitím kompilovaných C/C++ modulů frameworku GStreamer s přímým předáváním paměťových bufferů oproti režii zpracování obrazu a zápisu videa v jazyce Python na PC. Při hodnocení je zároveň nutné zohlednit dvě metodická omezení: zaprvé, prvek `queue leaky=downstream` v GStreameru při čtení ze souboru zahodil 156 mezisnímků (4 957 zpracovaných snímků oproti 5 113 snímkům v bezztrátové frontě na PC), a zadruhé, zatímco na GPU bylo možné jednotlivé chyby typu FP a FN manuálně verifikovat, u Hailo-8 byla vzhledem k efektu *ghostingu* a absenci snímkové anotace souřadnic matice záměn odvozena makroskopicky z koncových součtů tříd. Pro plně deterministické srovnání na úrovni jednotlivých snímků v budoucích pracích bude vhodné vyhodnotit sledování pomocí standardizovaných metrik MOTA a IDF1 nad snímkově anotovaným datasetem.

Z pohledu praktického nasazení nelze jednu platformu označit za univerzálně nadřazenou, neboť každá cílí na odlišný provozní scénář a nabízí specifické výhody:

- **Výhody referenčního GPU řešení (PC + RTX 2050):** Hlavní předností této platformy je vyšší statistická přesnost ve většině sledovaných metrik ($\text{Accuracy}_{\text{count}} = 99{,}11\ \%$, Precision $= 96{,}40\ \%$, $F1 = 95{,}96\ \%$), absence kvantizačního šumu díky výpočtům v plné přesnosti (FP32/FP16) a vysoká odolnost dvoufázového algoritmu ByteTrack vůči falešnému štěpení stop (*ghostingu*) při zákrytech. Toto řešení je proto vhodnější pro centrální analytické servery, stacionární dohledová pracoviště se stálým napájením nebo offline forenzní vyhodnocování dopravních záznamů, kde je prioritou maximální přesnost počítání a klasifikace bez ohledu na spotřebu energie (> 100 W) a rozměry hardwaru.
- **Výhody navrženého Edge řešení (Raspberry Pi 5 + Hailo-8):** Hlavní předností okrajového uzlu je více než desetinásobně nižší spotřeba elektrické energie ($\sim$8–12 W), kompaktní rozměry umožňující instalaci přímo do rozvaděče u kamery, nižší pořizovací náklady, vyšší snímková propustnost ($29{,}39$ FPS díky nativní GStreamer pipeline) a vysoká úplnost záchytu vozidel (Recall $= 98{,}21\ \%$). Navržené řešení navíc zpracovává obraz plně lokálně bez přenosu surového videa po síti v souladu s principy GDPR (*Privacy by Design*), což z něj činí ideální volbu pro plošné, autonomní a dlouhodobé nasazení v rámci dopravní infrastruktury chytrých měst.

6. Závěr

V první části práce byla zpracována teoretická rešerše zaměřená na moderní jednofázové detekční architektury (zejména rodinu modelů YOLO a porovnání jejích velikostních variant), algoritmy víceobjektového sledování (SORT, DeepSORT) a metody prostorového vyhodnocování průjezdů pomocí oblastí zájmu (ROI). Zvláštní pozornost byla věnována hardwarové akceleraci neuronových sítí pomocí specializovaných čipů (NPU) na vestavěných zařízeních a dodržení legislativních požadavků GDPR prostřednictvím konceptu *Privacy by Design*.

V rámci praktické části, která se skládá z návrhu a implementace systému, byla navržena a implementována modulární vícevrstvá architektura systému pro monitorování dopravy pomocí objektově orientovaného programování (OOP) a popsaná pomocí diagramů UML a ER. Tato architektura byla realizována na okrajovém zařízení Raspberry Pi 5 osazeném dedikovaným AI akcelerátorem Hailo-8 (s výkonem 26 TOPS) a současně na referenčním přenosném počítači s grafickou kartou (GPU) NVIDIA GeForce RTX 2050. Pro každé z těchto zařízení bylo potřeba připravit vhodný tvar neuronové sítě využité k detekci, klasifikaci a sledování objektů. Pro zajištění objektivního srovnání byl na obou platformách nasazen totožný detekční model `YOLOv8m`. Pro výpočetní uzel na hraně sítě byl vedle experimentálního ověření vlastního kompilačního procesu (exportu do meziformátu ONNX a 8bitové post-tréninkové kvantizace PTQ přes meziformát HAR v nástroji Hailo Dataflow Compiler) pro finální provoz nasazen plně optimalizovaný binární model `yolov8m.hef` v přesnosti INT8 z oficiální sady `hailo-rpi5-examples`, zatímco na GPU s podporou CUDA pracoval model v plné přesnosti (FP32/FP16).

Během řešení této bakalářské práce prošla rodina modelů YOLO velmi rychlou evolucí (postupně vyšly čtyři nové generace `YOLOv9` až `YOLOv12`), avšak v době zahájení implementace představoval model `YOLOv8m` jedinou plně odladěnou architekturu dostupnou pro akcelerátor Hailo-8. V současnosti však již oficiální úložiště Hailo Model Zoo [35] obsahuje předkompilované binární soubory HEF a kalibrační profily i pro novější rodiny `YOLOv9`, `YOLOv10` a `YOLOv11` (např. `yolov11s` či `yolov11m`). Díky modulárnímu návrhu vytvořené GStreamer pipeline lze tyto novější modely do systému nasadit pouhou výměnou konfigurační cesty k souboru HEF a příslušné sdílené knihovny post-processingu, což zaručuje dlouhodobou udržitelnost celého řešení.

Zpracování obrazového toku na okrajovém zařízení bylo realizováno pomocí vysokoúrovňového řetězce v multimediálním frameworku GStreamer, doplněného o geometrickou filtraci průjezdů v polygonální ROI zóně s využitím knihovny Shapely. Pro přenos anonymizovaných detekčních metadat byl nasazen asynchronní protokol MQTT, na který navazuje databázová vrstva s relační databází PostgreSQL (využívající fond spojení a dávkové vkládání), backendové REST API ve

frameworku FastAPI a interaktivní analytický dashboard vytvořený pomocí nástrojů Streamlit a Plotly. Součástí práce je rovněž kontejnerizace v prostředí Docker umožňující snadnou replikaci a simulaci systému.

Funkčnost, propustnost a přesnost celého řešení byly ověřeny na reálném dopravním videozáznamu pořízeném za zhoršených zimních povětrnostních podmínek (husté sněžení) při shodném nastavení prahových hodnot i polygonu ROI vůči manuálnímu sčítání průjezdů zónou (*ground truth*, 112 vozidel). Experimentální srovnání ukázalo specifické přednosti obou zkoumaných architektur. Referenční GPU řešení s modelem `YOLOv8m` v plné přesnosti a algoritmem ByteTrack dosáhlo vyšší statistické přesnosti ve většině hodnoticích metrik – zaznamenalo 111 objektů ($\text{Accuracy}_{\text{count}} = 99{,}11\ \%$), přesnost detekce $\text{Precision} = 96{,}40\ \%$ a F1-skóre $95{,}96\ \%$ (při 5 přehlédnutých a 4 falešně pozitivních detekcích) při průměrné propustnosti 25,64 FPS (39,00 ms na snímek). Navržené Edge řešení s akcelerátorem Hailo-8 naproti tomu vynikalo v provozní efektivitě a rychlosti zpracování – dosáhlo vyšší průměrné propustnosti 29,39 FPS (34,02 ms na snímek) při vytížení procesoru pouze 25–35 %, řádově nižší spotřeby energie ($\sim$8–12 W oproti > 100 W) a vyšší úplnosti záchytu $\text{Recall} = 98{,}21\ \%$ (při F1-skóre $92{,}05\ \%$ a $\text{Accuracy}_{\text{count}} = 86{,}61\ \%$ pro 127 detekovaných objektů, kde odchylka vznikla vícenásobným započítáním vozidel při zákrytu za sloupem).

Hlavním praktickým přínosem práce je vytvoření plně autonomního, finančně dostupného a energeticky úsporného monitorovacího uzlu, který nevyžaduje přenos citlivého videozáznamu do cloudu a lze jej díky automatizačním a kontejnerovým skriptům okamžitě nasadit v reálné dopravní infrastruktuře.

V budoucnu lze vytvořený systém dále rozšířit několika směry:

- **Vlastní dotrénování modelu (*fine-tuning*):** Dotrénování modelu YOLOv8m na specifickém lokálním datasetu z českých komunikací v zimních podmínkách za účelem zvýšení jistoty detekce částečně zakrytých vozidel a eliminace občasných záměn mezi dodávkami a osobními automobily.
- **Experimentální srovnání dalších detekčních architektur:** Rozšíření komparativní analýzy mezi akcelerátorem Hailo-8 a referenční GPU o novější generace rodiny YOLO (např. `YOLOv10`, `YOLOv11`) i alternativní jednofázové detektory typu SSD (*Single Shot MultiBox Detector*, např. `MobileNet-SSD`), a vyhodnocení jejich poměru mezi snímkovou propustností, přesností sčítání a odolností vůči 8bitové kvantizaci.
- **Pokročilá správa trajektorií a vektoru směru:** Rozšíření aplikační logiky v zóně ROI o vyhodnocování směrových vektorů pohybu (rozlišení vjezdů a výjezdů na jednotlivých ramenech křižovatky) a zpřesnění filtrace kandidátních stop pro potlačení efektu ghostingu.
- **Horizontální škálování:** Využití připravených databázových atributů (`device_name`, `model_name`) pro zapojení většího počtu okrajových jednotek Raspberry Pi z různých lokalit města do jednoho centrálního analytického dashboardu.

Seznam použitých zdrojů

1. SZELISKI, Richard. *Computer Vision: Algorithms and Applications*. 2. vyd. Cham, Switzerland: Springer, 2022. ISBN 978-3-030-34372-9.
2. ZHOU, Wei; YANG, Li; ZHAO, Lei; ZHANG, Runyu; CUI, Yifan; HUANG, Hongpu; QIE, Kun; WANG, Chen. Vision Technologies with Applications in Traffic Surveillance Systems: A Holistic Survey. 2025. Dostupné z arXiv: 2412.00348 [cs.CV].
3. REDMON, Joseph; DIVVALA, Santosh; GIRSHICK, Ross; FARHADI, Ali. You Only Look Once: Unified, Real-Time Object Detection. 2016, 779–788. Dostupné z DOI: 10.1109/CVPR.2016.91.
4. BEWLEY, Alex; GE, Zongyuan; OTT, Lionel; RAMOS, Fabio; UPCROFT, Ben. Simple Online and Realtime Tracking. *2016 IEEE International Conference on Image Processing (ICIP)*. 2016, 3464–3468. Dostupné z DOI: 10.1109/ICIP.2016.7533003.
5. PEREIRA, Ricardo; CARVALHO, Guilherme; GARROTE, Luís; NUNES, Urbano J. Sort and Deep-SORT Based Multi-Object Tracking for Mobile Robotics: Evaluation with New Data Association Metrics. *Applied Sciences*. 2022, **12**(3), 1319. Dostupné z DOI: 10.3390/app12031319.
6. WOJKE, Nicolai; BEWLEY, Alex; PAULUS, Dietrich. Simple online and realtime tracking with a deep association metric. In: *2017 IEEE International Conference on Image Processing (ICIP)*. 2017, s. 3645–3649. Dostupné z DOI: 10.1109/ICIP.2017.8296962.
7. ZHANG, Yifu; SUN, Peize; JIANG, Yi; YU, Dongdong; WENG, Fucheng; YUAN, Zehuan; LUO, Ping; LIU, Wenyu; WANG, Xinggang. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. In: *Computer Vision – ECCV 2022*. Springer Nature Switzerland, 2022, s. 1–21. Dostupné z DOI: 10.1007/978-3-031-20047-2_1.
8. LIN, Tsung-Yi; MAIRE, Michael; BELONGIE, Serge; HAYS, James; PERONA, Pietro; RAMANAN, Deva; DOLLÁR, Piotr; ZITNICK, C. Lawrence. Microsoft COCO: Common Objects in Context. In: *Lecture Notes in Computer Science* [online]. Cham: Springer International Publishing, 2014, s. 740–755 [cit. 2026-04-15]. ISBN 978-3-319-10601-4. ISSN 0302-9743. Dostupné z DOI: 10.1007/978-3-319-10602-1_48.
9. TERVEN, Juan; CÓRDOVA-ESPARZA, Diana-Margarita; ROMERO-GONZÁLEZ, Julio-Alejandro. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction*. 2023, **5**(4), 1680–1716. ISSN 2504-4990. Dostupné z DOI: 10.3390/make5040083.

10. JOCHER, Glenn; CHAURASIA, Ayush; QIU, Jing. *Ultralytics YOLOv8 Documentation*. 2023. Dostupné také z: https://docs.ultralytics.com/models/yolov8/. [Online; cit. 2026-10-28].

11. GS INDUSTRIES. *Security cameras watching from above to everywhere* [https://www.gsindustries.com.au/news/cameras-on-top-of-street-light-poles/]. [B.r.]. [Online; cit. 2026-10-13].

12. LI, En; ZENG, Liekang; ZHOU, Zhi; CHEN, Xu. Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing. *IEEE Transactions on Wireless Communications*. 2020, **19**(1), 447–457. Dostupné z DOI: 10.1109/TWC.2019.2946140.

13. HAILO TECHNOLOGIES LTD. *Automatic License Plate Recognition with Hailo Processors* [online]. Hailo Blog, 2024. [cit. 2025-12-04]. Dostupné z: https://hailo.ai/ja/blog/automatic-license-plate-recognition-with-hailo-processors/.

14. AKIRA AI. *Edge Computing vs. Cloud Computing* [online]. 2025. [cit. 2026-02-05]. Dostupné z: https://www.akira.ai/insights/edge-computing-vs-cloud-computing/.

15. HAILO TECHNOLOGIES LTD. *Dataflow AI Architecture & Chip Design*. 2026. Dostupné také z: https://hailo.ai/products/technology/. [Online; cit. 2026-10-14].

16. RASPBERRY PI FOUNDATION. *Raspberry Pi Documentation* [online]. Cambridge, 2024 [cit. 2025-11-03]. Dostupné z: https://www.raspberrypi.com/documentation/.

17. RASPBERRY PI FOUNDATION. *AI Kit and AI HAT+ Software* [online]. [cit. 2025-12-04]. Dostupné z: https://www.raspberrypi.com/documentation/computers/ai.html.

18. RPISHOP.CZ. *Rozšiřující modul Raspberry Pi AI HAT+ s integrovaným procesorem Hailo-8* [online]. 2025. [cit. 2026-02-07]. Dostupné z: https://rpishop.cz/554037/raspberry-pi-ai-hat-26-tops/.

19. GSTREAMER TEAM. *GStreamer: Open Source Multimedia Framework Documentation*. 2025. Dostupné také z: https://gstreamer.freedesktop.org/documentation/. [Online; cit. 2026-10-30].

20. NAMATĒVS, Ivars; KADIĶIS, Roberts; ZENCOVS, Anatolijs; LEJA, Laura; DOBRĀJS, Artis. Dataset of Annotated Virtual Detection Line for Road Traffic Monitoring. *Data*. 2022, **7**(4). ISSN 2306-5729. Dostupné z DOI: 10.3390/data7040040.

21. PYTHON SOFTWARE FOUNDATION. *Python 3.11 Documentation* [online]. 2025. [cit. 2025-11-03]. Dostupné z: https://docs.python.org/3.11/.

22. HAILO TECHNOLOGIES LTD. *TAPPAS User Guide* [online]. 3.31.0. vyd. Hailo, 2024. [cit. 2026-02-03]. Dostupné z: https://github.com/hailo-ai/tappas. Software documentation.

23. GILLIES, Sean et al. *Shapely: Manipulation and Analysis of Geometric Objects in the Cartesian Plane*. 2025. Dostupné také z: https://shapely.readthedocs.io/. [Online; cit. 2026-10-30].

24. OASIS OPEN. *MQTT Version 5.0* [online]. 2019. [cit. 2025-11-04]. Dostupné z: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

25. PSYCOPG TEAM. *Psycopg Documentation: PostgreSQL Database Adapter for Python* [online]. 2025. [cit. 2025-11-04]. Dostupné z: https://www.psycopg.org/.

26. RAMÍREZ, Sebastián. *FastAPI Documentation*. 2025. Dostupné také z: https://fastapi.tiangolo.com/. [Online; cit. 2026-10-30].

27. MCKINNEY, Wes. Data Structures for Statistical Computing in Python. 2010, 56–61. Dostupné z DOI: 10.25080/Majora-92bf1922-00a.

28. HARRIS, Charles R.; MILLMAN, K. Jarrod; WALT, Stéfan J. van der et al. Array Programming with NumPy. *Nature*. 2020, 357–362. Dostupné z DOI: 10.1038/s41586-020-2649-2.

29. SNOWFLAKE INC. *Streamlit Documentation: Faster Way to Build and Share Data Apps*. 2025. Dostupné také z: https://docs.streamlit.io/. [Online; cit. 2026-10-30].

30. PLOTLY TECHNOLOGIES INC. *Collaborative Data Science* [online]. Montréal, QC, 2015 [cit. 2025-11-04]. Dostupné z: https://plot.ly.

31. BRADSKI, Gary. The OpenCV Library. *Dr. Dobb's Journal of Software Tools*. 2000, **25**(11), 120–125.

32. DOCKER INC. *Docker Documentation*. 2025. Dostupné také z: https://docs.docker.com/. [Online; cit. 2026-10-30].

33. HAILO TECHNOLOGIES LTD. *Hailo Dataflow Compiler User Guide* [online]. 2024. [cit. 2026-03-01]. Dostupné z: https://hailo.ai.

34. HAILO TECHNOLOGIES LTD. *Hailo Raspberry Pi 5 Examples* [online]. 2024. [cit. 2026-03-01]. Dostupné z: https://github.com/hailo-ai/hailo-rpi5-examples.

35. HAILO TECHNOLOGIES LTD. *Hailo Model Zoo* [online]. 2024. [cit. 2026-03-01]. Dostupné z: https://github.com/hailo-ai/hailo_model_zoo.

A. Externí přílohy

Zdrojové kódy navrženého systému, instalační a konfigurační skripty, kontejnerové definice, dokumentace architektury, vyexportované výsledky měření i samotný text této bakalářské práce jsou uloženy v přiloženém elektronickém archivu a verzovány ve veřejném GitHub repozitáři dostupném na adrese: `https://github.com/imang212/Bachelor_Thesis`.

Detailní postupy instalace a konfigurace obou testovaných výpočetních platforem jsou vyčleněny do samostatných adresářů elektronické přílohy a opírají se o oficiální dokumentaci výrobců:

- **Konfigurace a instalace okrajového zařízení (Raspberry Pi 5 + Hailo-8):** Nachází se v adresáři `rpi_src/`. Zahrnuje automatizační skript `autosetup_hailo_rpi5.sh` pro přípravu 64bitového systému Raspberry Pi OS (Debian Bookworm) [16], aktivaci sběrnice PCIe Gen 3, instalaci ovladačů modulu AI HAT+ [17], prostředí HailoRT, knihoven TAPPAS [22] a multimediálního frameworku GStreamer [19]. Skript `autorun.sh` následně zajišťuje automatické spouštění detekční pipeline, databázového odběratele a webového rozhraní po startu systému (doplněné o manuál v `rpi_src/pipeline/script-guide.md`).
- **Konfigurace a instalace referenční stanice (PC + NVIDIA GPU):** Nachází se v adresáři `pc_src/` (pro inferenci v prostředí CUDA s knihovnami Ultralytics [10], OpenCV [31] a algoritmem ByteTrack [7] dle `pc_src/requirements.txt` a `pc_src/docker-compose.yml`) a v adresáři `rpi_src/model_compile/` (pro kompilaci a 8bitovou kvantizaci modelů do formátu HEF pomocí nástrojů Hailo Dataflow Compiler [33] a Hailo Model Zoo [35]).

Struktura všech hlavních adresářů a souborů přiloženého úložiště je popsána v tabulce A.1.

Cesta v úložišti	Popis obsahu a funkce
`.github/workflows/`	Definice automatizované CI/CD pipeline (`ci.yml`) v prostředí GitHub Actions zajišťující spouštění jednotkových testů (`pytest`), validaci Docker Compose konfigurací a kontrolu Kubernetes manifestů.
`common/`	Sdílený Python balíček obsahující centralizované načítání konfigurace z proměnných prostředí (`config.py`), správu databázového poolu PostgreSQL (`DatabaseManagerPostgre.py`) a komunikačního MQTT klienta (`MQTTClient.py`).
`rpi_src/`	Zdrojové kódy pro okrajové zařízení Raspberry Pi 5 s akcelerátorem Hailo-8: instalační a spouštěcí skripty (`autosetup_hailo_rpi5.sh`, `autorun.sh`), GStreamer pipeline, FastAPI backend (`API_with_streamlit.py`) a Streamlit dashboard (`streamlit_dashboard.py`) a průvodce (`script-guide.md`) ve složce `pipeline/`, kompilace modelů v `model_compile/`, SQL inicializace v `db/` a kontejnerová simulace v `docker/`.
`pc_src/`	Skripty pro referenční vyhodnocení na osobním počítači s GPU (`yoloTrafficDetectionSystem_PC_GPU.py`, `traffic_stats.py`, konfigurace `bytetrack.yaml`) včetně definice kontejnerového a Kubernetes nasazení (`Dockerfile`, `docker-compose.yml`, `k8s/`).
`videos/`	Testovací dopravní videozáznam (`sample_video.mp4`, spravovaný pomocí Git LFS) využitý pro experimentální srovnání obou platforem.
`results/`	Výstupní data z experimentálního testování, telemetrické a statistické soubory ve formátu JSON pro obě platformy a skripty pro generování obrazových výřezů.
`tests/`	Sada automatizovaných jednotkových testů (`pytest`) ověřujících konfigurační logiku, přihlašování (MQTT, Databáze) a připojování (IP kamera), geometrickou filtraci zóny ROI a sdílené moduly.
`docs/`	Technická dokumentace projektu obsahující architektonická schémata (`diagrams/`) a zdrojové Python skripty pro generování UML a ER diagramů (`diagrams/scripts/`) ve formátu Mermaid.
`thesis_text/`	Zdrojové kódy textu bakalářské práce v systému LaTeX (`Bachelor_Thesis_CS.tex`), rozdělené kapitoly (`chapters/cs/`), bibliografie (`references.bib`), složku s prostudovanou literaturou a grafické přílohy (`images/`).
`.env.example`	Šablona konfiguračního souboru pro nastavení proměnných prostředí (databáze, MQTT broker, RTSP stream) bez citlivých hesel.

Tabulka A.1.: Struktura přiloženého elektronického úložiště projektu.